Blog Application - CI/CD Deployment Report

A three-tier blog application with a React frontend, Express API and persistent MySQL database on Amazon EKS. The implementation integrates automated testing, code-quality analysis, container scanning, dual-registry publication, rolling deployment and monitoring in AWS Mumbai.

Project reference	Verified value
Application URL	https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com
GitHub repository	https://github.com/praveenpreni/BlogReact
Document URL	https://github.com/praveenpreni/BlogReact/blob/main/docs/Cloud_Execution_Report.pdf
Release build	Jenkins BlogApp/main #14 - SUCCESS; 89.04 seconds
Exact execution source	00bfc302acd73933912fd12579b72885081d22cb
Immutable image tag	00bfc302acd7-14
Private Jenkins build URL	http://127.0.0.1:18080/job/BlogApp/job/main/14/

Validation outcomes: 10 tests passed; quality gate passed; both container image gates passed; images published to both registries; public write/read/comment validation passed. The rolling deployment test served 196 / 196 HTTP 200 responses.

Screenshots are genuine browser captures, including live command execution views. Tables reproduce attached command/API results. UTC timestamps in raw records and Asia/Kolkata timestamps in browser views refer to the same execution. Administrative localhost links require the operator's authenticated forwarding session.

Deployment validation overview

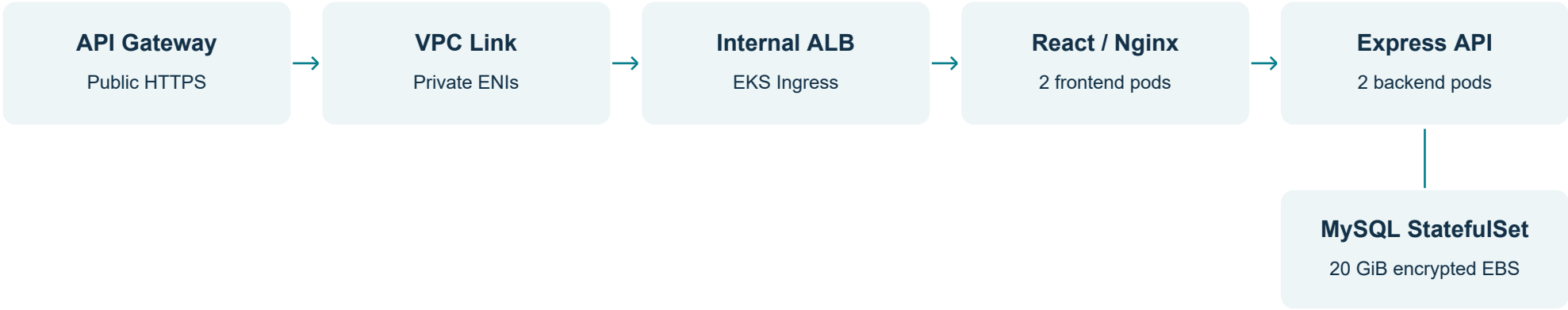
The deployment was validated through pipeline execution, live Kubernetes commands, AWS console inspection and public application requests. Build #14 identifies the tested release. Each validation run is recorded with its own timestamp and result.

Validation area	Execution evidence
Network isolation	Live get networkpolicy; unrelated client exits 1; frontend readiness and backend-to-MySQL succeed.
Configuration / persistent storage	Live get configmap,secret,pvc,statefulset, with Secret names/counts only.
External access	Live Services/Ingress; internal ALB, API Gateway integration and Available VPC Link console pages.
Deployed image versions	Live Deployment and pod image tags/digests after redeploying versioned immutable build #14 images.
Production practices	Actual describe deployment backend: replicas, RollingUpdate, probes, requests and limits.
Real Trivy gate	Jenkins #14: both application image scans exit 0; archived JSON reports contain zero HIGH/CRITICAL findings.
Rolling / database recreation	Image redeployment: 96/96 HTTP 200; MySQL pod recreation retains PVC, post and comment.
Live logging	Fresh kubectl logs for all three tiers, streamed into the live operator command console.

Command screenshots use a loopback browser console that runs kubectl subprocesses and streams their output. Evidence includes command timestamps, exit codes and raw output files. AWS console and Jenkins screenshots identify the deployed resources and pipeline results.

Three tiers with a private application origin

Internet clients use an AWS-managed HTTPS endpoint. API Gateway reaches the internal ALB through a VPC Link. Kubernetes Services connect frontend, backend and MySQL. The private API Gateway-to-ALB hop uses HTTP; public HTTPS terminates at API Gateway.



Layer	Executed controls
EKS / workers	Kubernetes 1.35; two private t3.large workers; two Availability Zones; IMDSv2; encrypted disks.
Platform services	VPC CNI NetworkPolicy enforcement, EBS CSI, metrics-server, AWS Load Balancer Controller.
Private CI host	t3.xlarge with encrypted 100 GiB disk; no inbound security-group rules; SSM administration.
CI tools	Jenkins 2.580.1; SonarQube Community Build 26.10.0.133078 + PostgreSQL 17; Nexus 3.97.2.
Observability	Prometheus/Grafana/Alertmanager on persistent storage; Kubernetes application logs; CloudWatch EKS API/audit/authenticator logs.

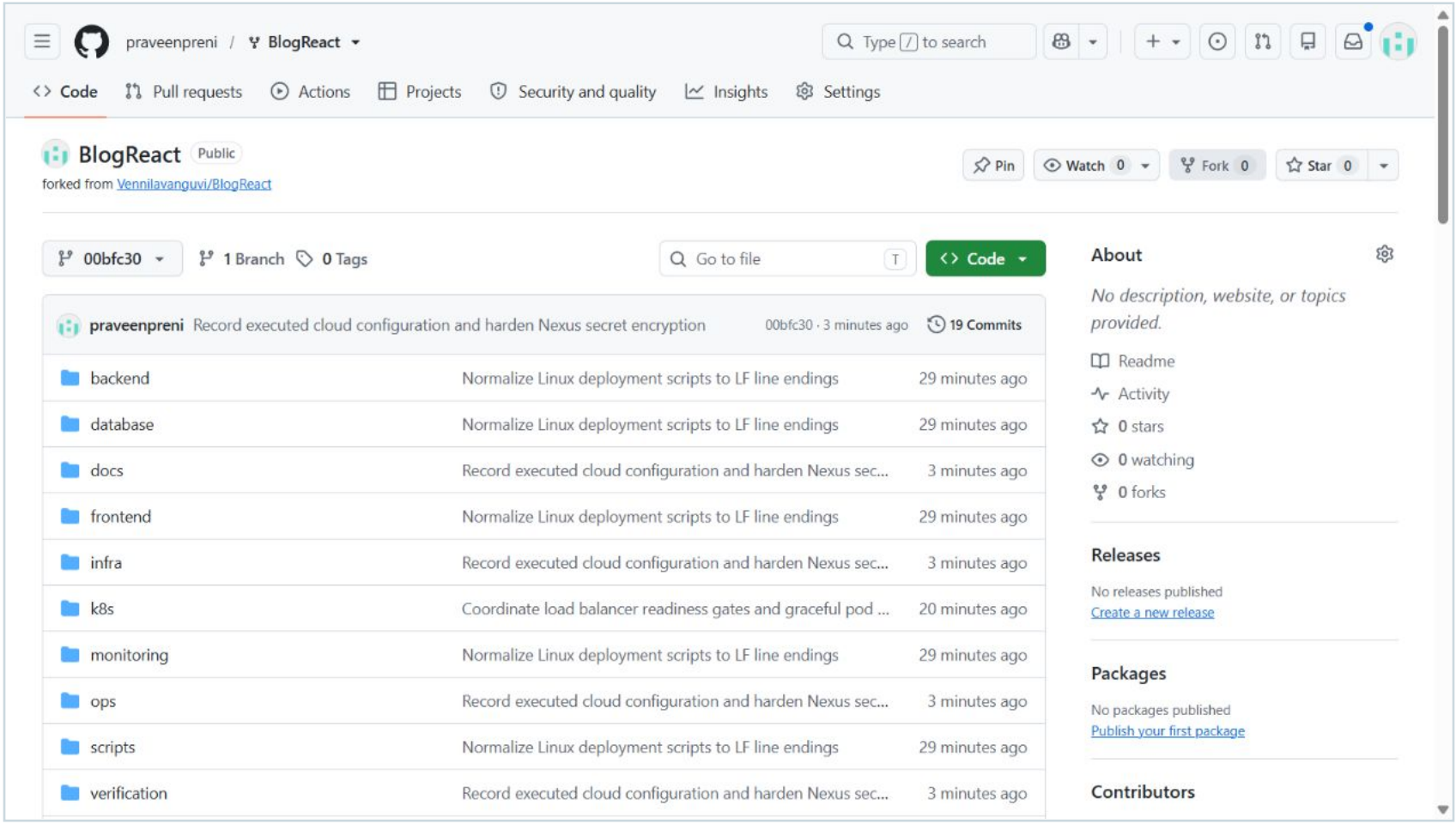
A custom domain was unnecessary. CloudFront distribution creation was blocked by AWS account verification; API Gateway provides the executed HTTPS path. The unused CloudFront origin was removed.

Requirement-to-evidence map

Brief	Requirement	Evidence	Recorded result
1	Working three-tier application	E08	Public register/login/post/read/comment operations passed.
2	Production-oriented Docker builds	E02	Multi-stage images, .dockerignore, non-root users, health checks; actual runtime inspection.
3	Persistent and secure database	E03, E09, E15	MySQL StatefulSet; encrypted PVC; schema initialization; recreation and restore verified.
4	Nexus and ECR	E05	Both versioned frontend/backend images present in both registries and used by EKS.
5	SonarQube and gate enforcement	E04	Current analysis passed; isolated missing-coverage fixture rejected before deployment.
6	Trivy image scanning	E06	HIGH/CRITICAL gate passed after remediation; vulnerable fixture rejected.
7	EKS deployment resources	E07	Deployments, StatefulSet, Services, Ingress, ConfigMap, Secret and PVC running.
8	Kubernetes production practices	E02, E07, E10	Replicas, probes, resources, service discovery, PDB/HPA and ALB readiness gates.
9	Configuration and secrets	E11	ConfigMap + Secret references; generated values retained privately in Secrets Manager.
10	Automated CI/CD	E03	Checkout, test, analysis, gates, build, publish, rollout and smoke validation succeeded.
11	Rolling deployment	E10	Commit-triggered release #12: 196 sampled requests, zero errors.
12	Prometheus / Grafana	E13	All six dashboard queries returned data; controlled alert fired and cleared.
13	Application logging	E14	Frontend, backend and MySQL kubectl log exports, plus EKS control-plane logging.
14	Security practices	E02, E06, E11, E12	Scoped IAM/RBAC, network denial tests, non-root execution and credential-free source.

Source repository and release traceability

Release source: 00bfc302acd73933912fd12579b72885081d22cb
Baseline attribution: Vennilavanguvi/BlogReact, commit 420de1b1508d45addc2e76a3e9f24df274459bba. The repository screenshot is pinned to the tested source commit.

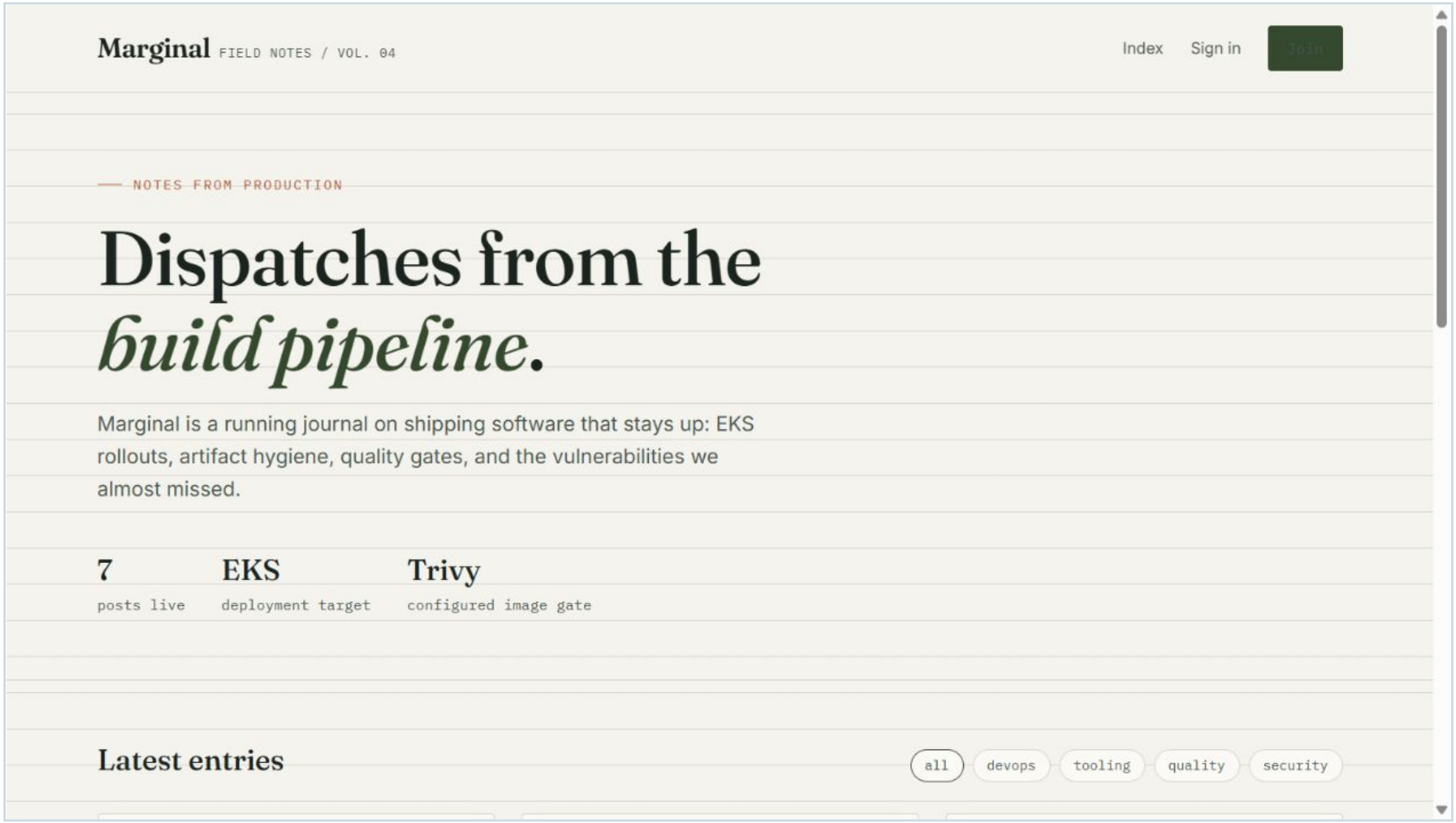


Pinned source view: https://github.com/praveenpreni/BlogReact/tree/00bfc302acd73933912fd12579b72885081d22cb

Release traceability links build #14 to its source commit, immutable image tag and container digests. Documentation commits are tracked separately from the application release.

Public HTTPS application

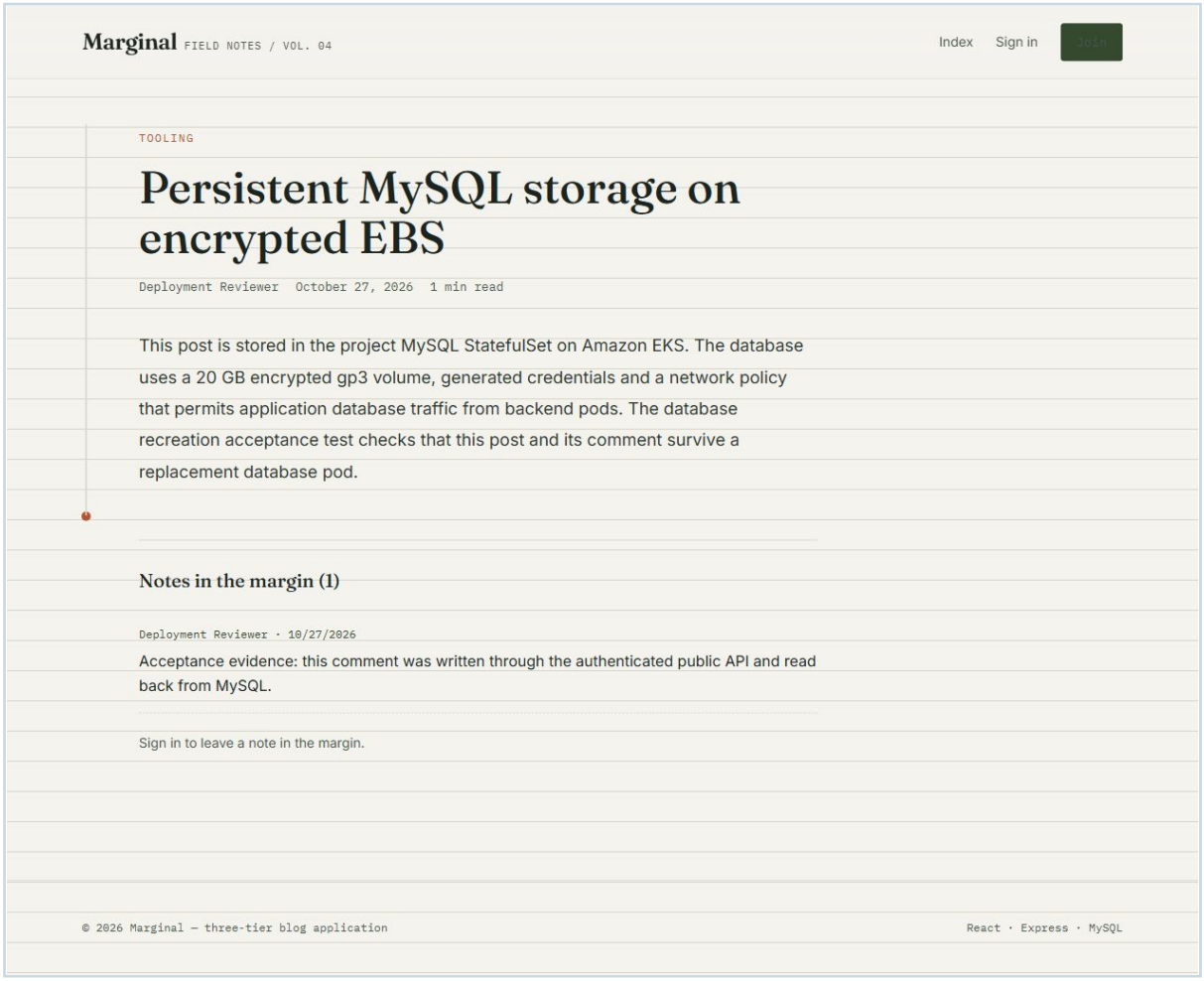
The deployed frontend loads data from the Express API and MySQL. Public API validation exercised registration, login, authenticated post creation, reading and comments. A generated test account was kept private.



Live application: https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com. Initial API proof: raw/E08-application-uat.json.

Persisted post and comment read through the frontend

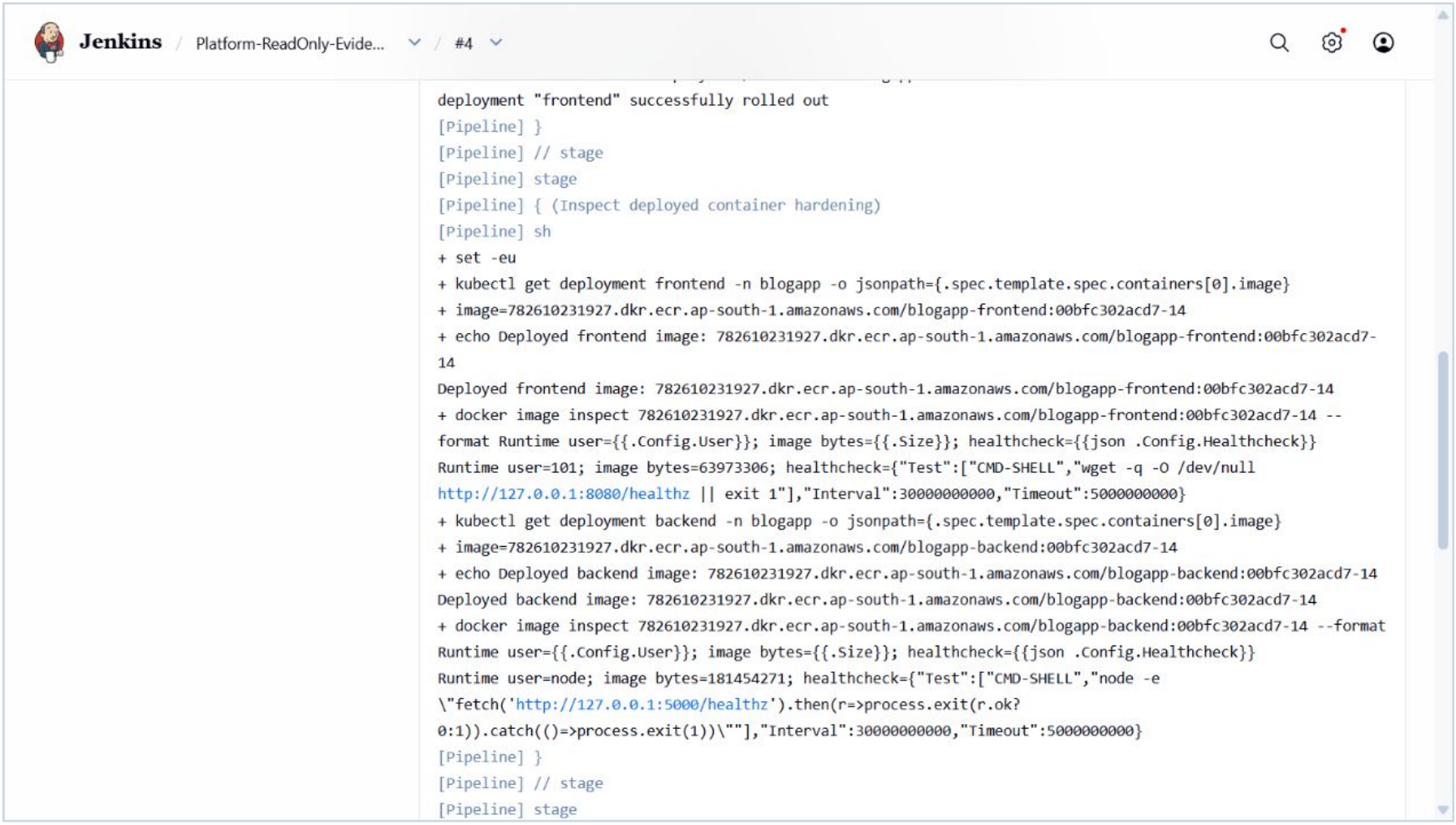
This page is rendered from data returned by the backend and stored in MySQL. The comment was created with an authenticated public API request. The same post and comment were verified after database pod recreation.



Post id 3 and its existing comment were retained. Persistence record: raw/E09-database-persistence-test.json.

Built images run as non-root users

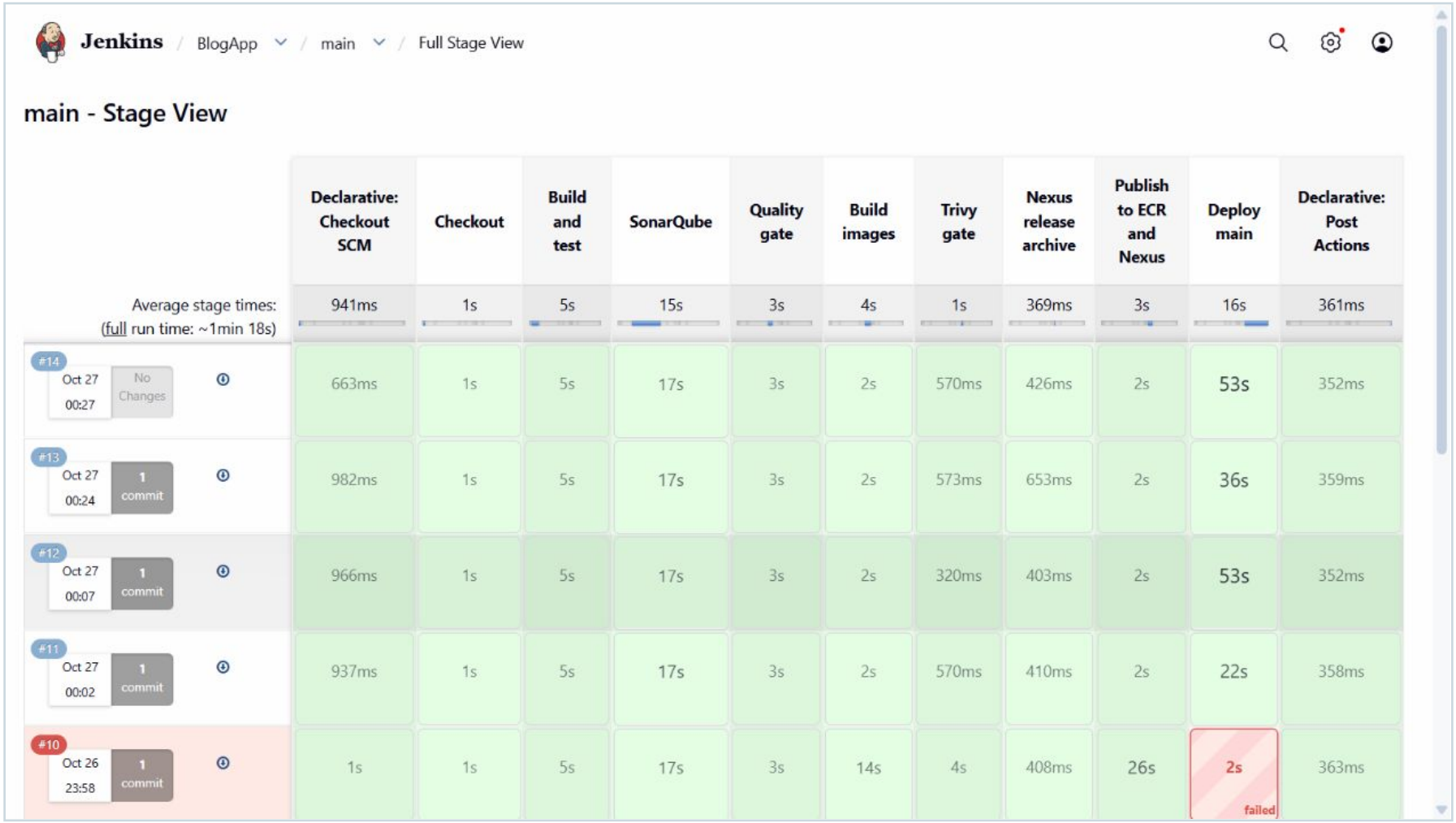
Frontend and backend use separate build/runtime stages, .dockerignore files and Docker health checks. Runtime packages were patched; unused Node package managers were removed from the backend image. Actual user IDs were frontend 101, backend 1000 and MySQL 999.



Real Jenkins image inspection for the deployed tag 00bfc302acd7-14. Uncompressed image sizes: frontend 63,973,306 bytes; backend 181,454,271 bytes.

All delivery stages succeeded

Build #14 completed every delivery stage successfully. The multibranch job automatically indexes source changes. Build #12 provides the commit-triggered rolling deployment availability measurement.

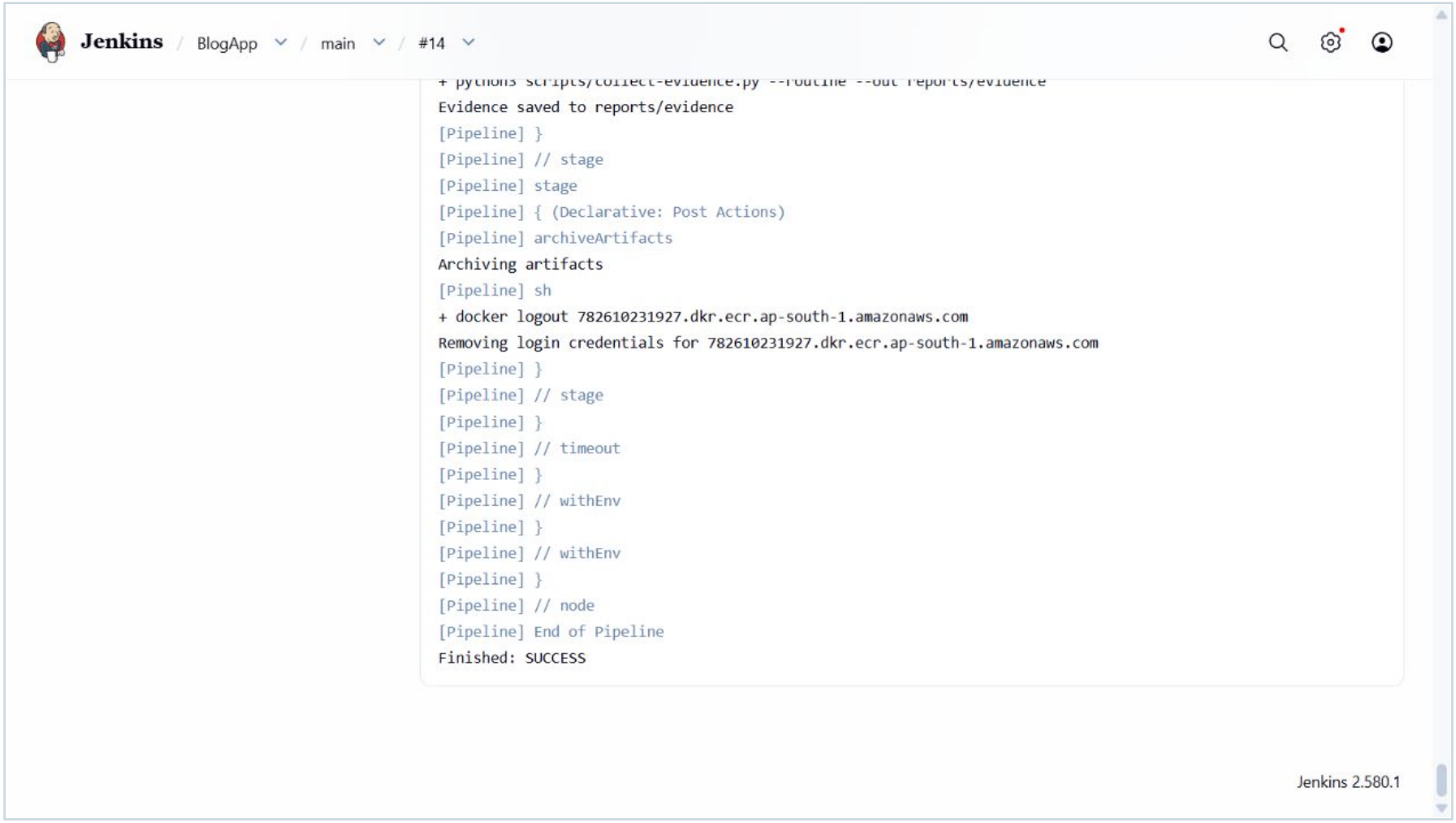


	Declarative: Checkout SCM	Checkout	Build and test	SonarQube	Quality gate	Build images	Trivy gate	Nexus release archive	Publish to ECR and Nexus	Deploy main	Declarative: Post Actions
Average stage times: (full run time: ~1min 18s)	941ms	1s	5s	15s	3s	4s	1s	369ms	3s	16s	361ms
#14 Oct 27 00:27 No Changes	663ms	1s	5s	17s	3s	2s	570ms	426ms	2s	53s	352ms
#13 Oct 27 00:24 1 commit	982ms	1s	5s	17s	3s	2s	573ms	653ms	2s	36s	359ms
#12 Oct 27 00:07 1 commit	966ms	1s	5s	17s	3s	2s	320ms	403ms	2s	53s	352ms
#11 Oct 27 00:02 1 commit	937ms	1s	5s	17s	3s	2s	570ms	410ms	2s	22s	358ms
#10 Oct 26 23:58 1 commit	1s	1s	5s	17s	3s	14s	4s	408ms	26s	2s failed	363ms

Actual Jenkins full stage view. Test, SonarQube, gate, Docker build, Trivy, Nexus archive, dual registry publishing and EKS deployment are green for #14.

Deployment and smoke validation completed

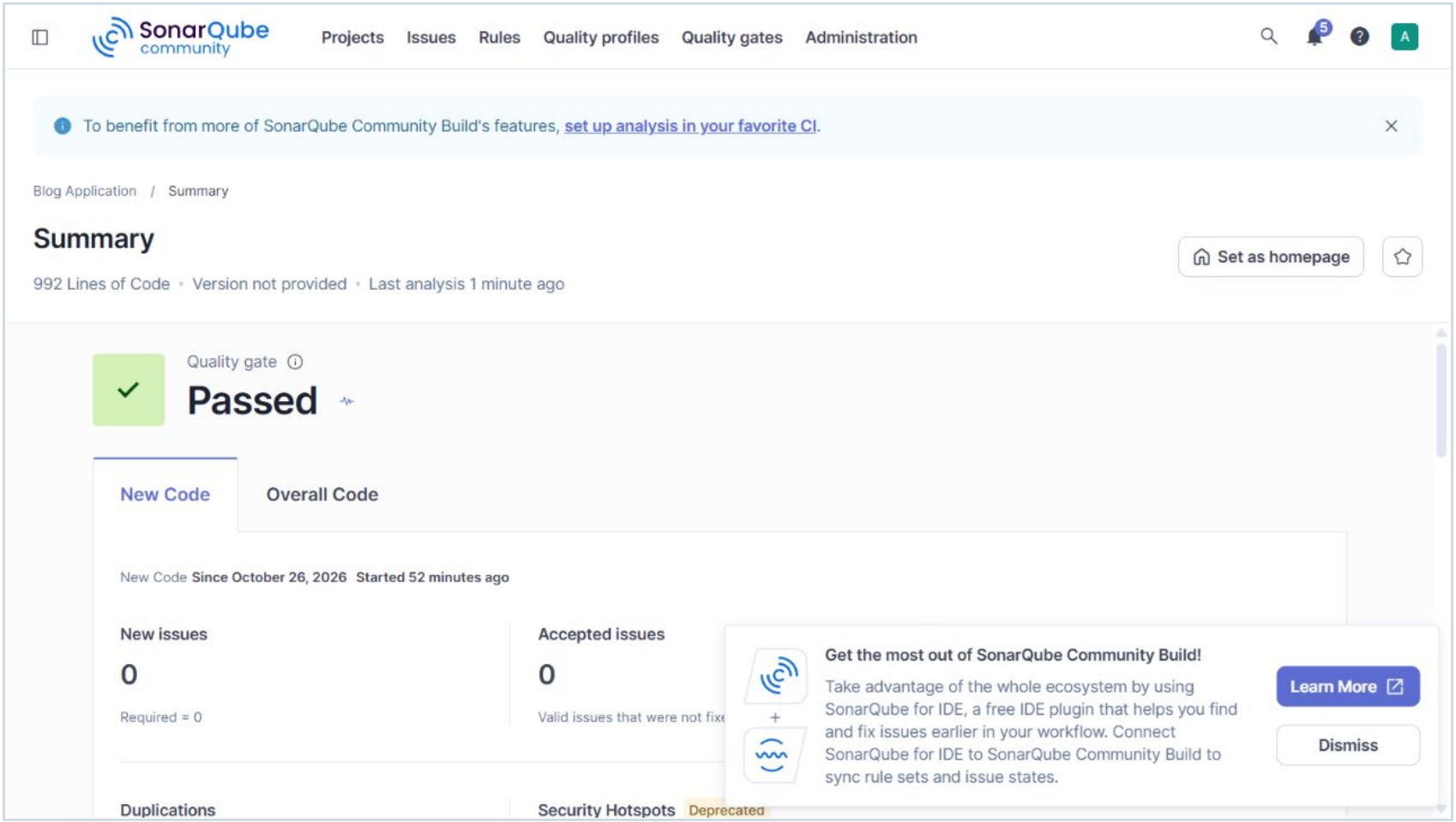
The console records rollout completion, public API write/read/comment checks, evidence collection and the final SUCCESS result. Raw console: raw/E03-jenkins-build-14-console.txt. Private operator build link: http://127.0.0.1:18080/job/BlogApp/job/main/14/



Actual #14 console final result. Unit tests: backend 7 + frontend 3; production frontend bundle built successfully.

Quality gate enforced for the specific analysis

The scanner communicates with Community Build 26.10.0.133078. The pipeline polls the submitted analysis ID and rejects a non-OK gate before image publication or deployment. Jenkins stays private without an incoming webhook.



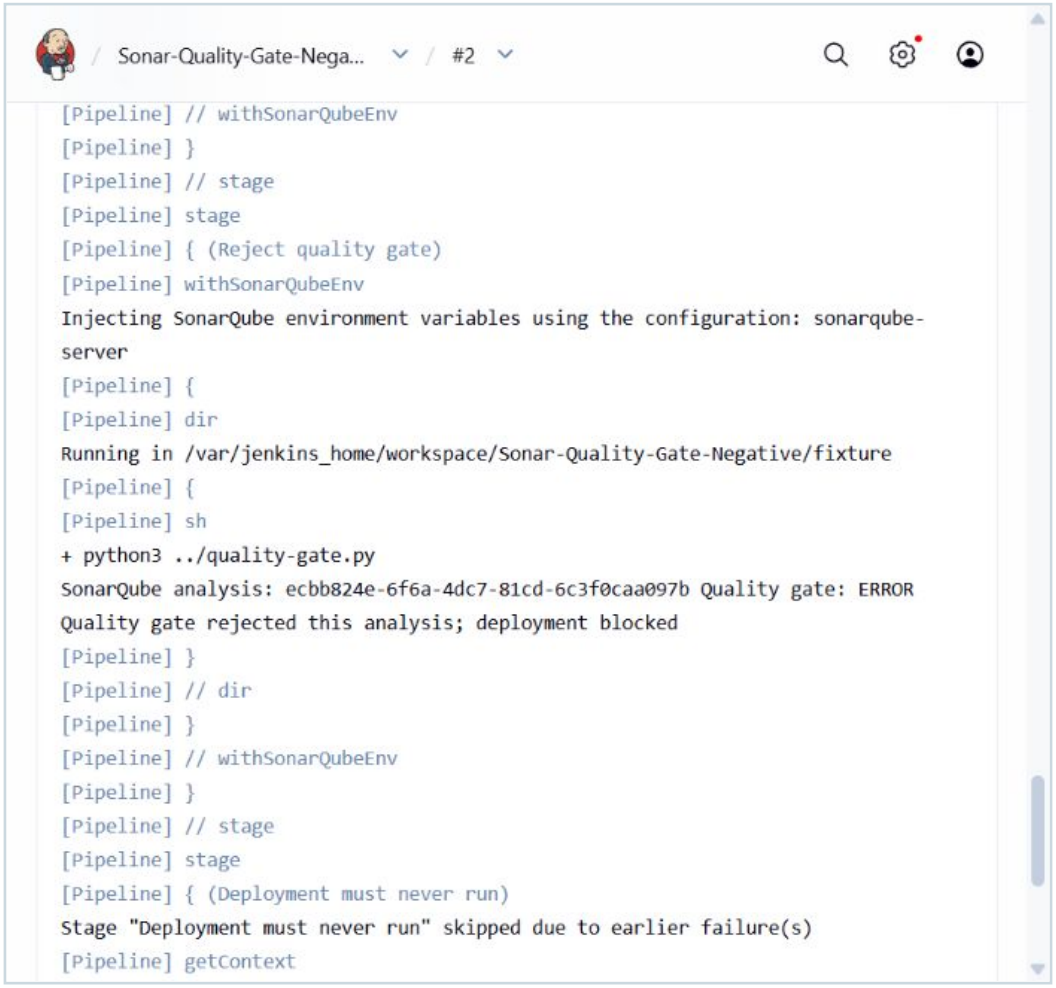
Main project new-code gate: Passed, zero new issues. Raw final analysis result: raw/E04-final-sonar-gate.json.

Limits: overall coverage is 40.6%; existing overall reliability rating C (21 findings), maintainability A (3 findings), security A (0 findings). No findings were waived. The gate is scoped to new code; Community Build has limited injection-vulnerability analysis.

E04 / NEGATIVE TEST

A failed quality gate blocks deployment

An isolated project intentionally omitted tests and used a separate coverage gate requiring 100%. It reported 0% coverage and ERROR. This did not modify or weaken the application project gate. The deployment stage was skipped.

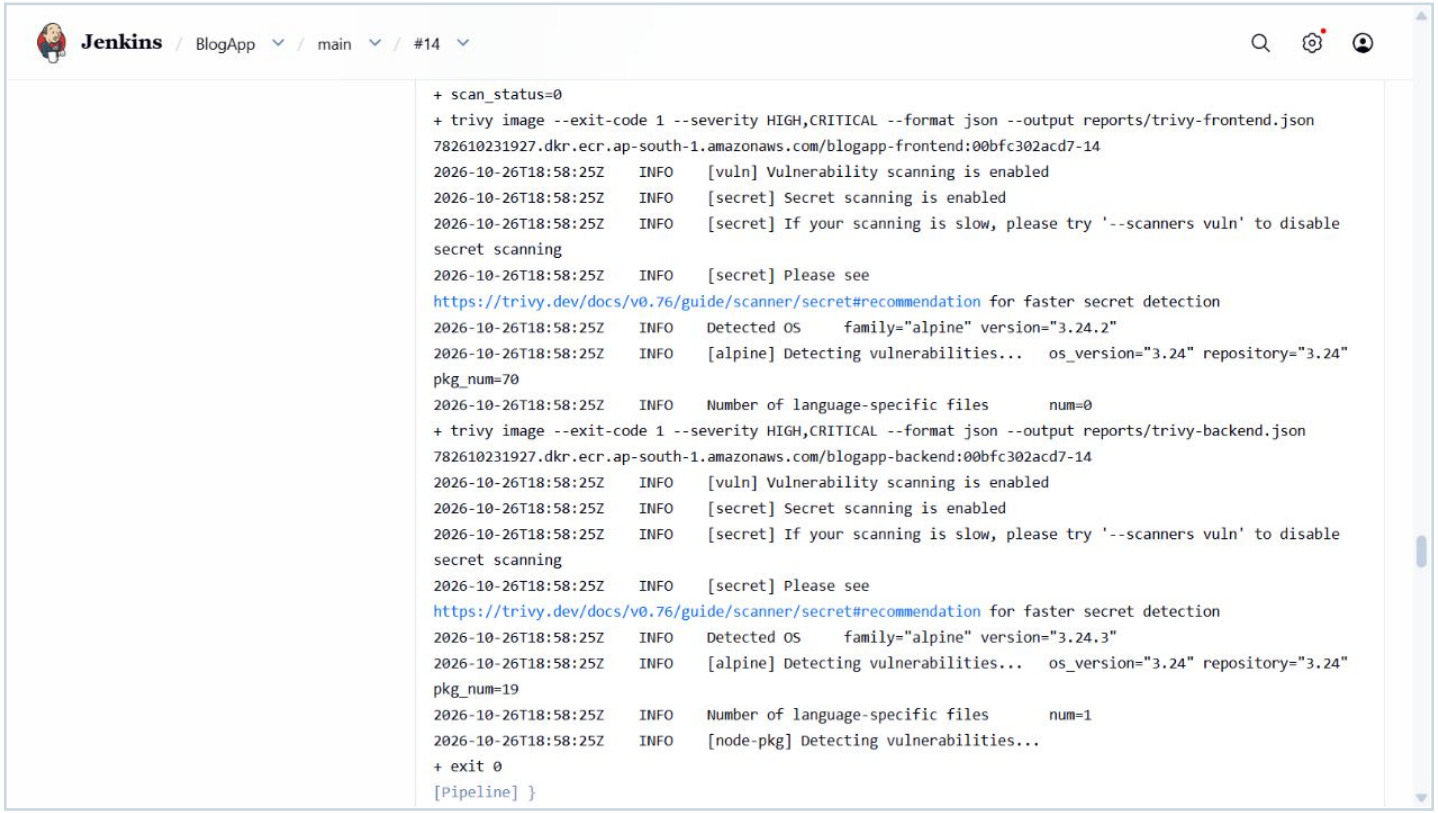


Sonar-Quality-Gate-Negative #2. Source definition: verification/cloud/SonarNegative.Jenkinsfile. Execution result: raw/E04-sonar-negative-result.json.

Container vulnerability scan results

Image	HIGH vulnerabilities	CRITICAL vulnerabilities	HIGH/CRITICAL scan gate
frontend	0	0	PASS
backend	0	0	PASS

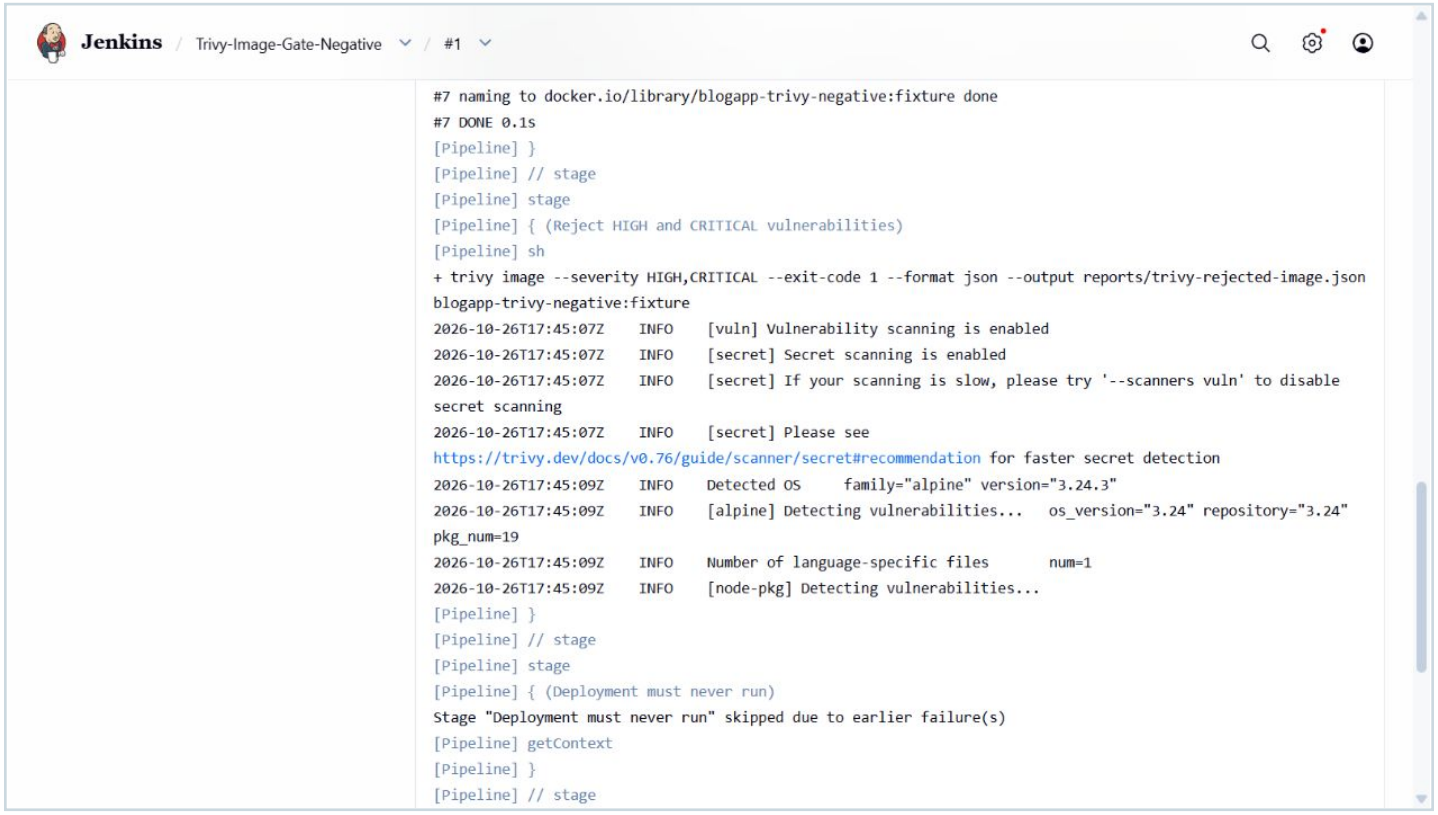
Trivy image scanning used --severity HIGH,CRITICAL --exit-code 1 before repository publication and EKS deployment. Runtime hardening includes patched Alpine libraries and removal of unused bundled package-manager dependencies. No --ignore-unfixed or severity downgrade was used.



Jenkins #14 console: frontend and backend tag 00bfc302acd7-14 scanned with HIGH,CRITICAL and exit-code 1; the stage ends with exit 0. JSON format writes findings to artifacts rather than printing a severity table. The zero counts above are independently verified from those exact archived reports.

A deliberately vulnerable image was rejected

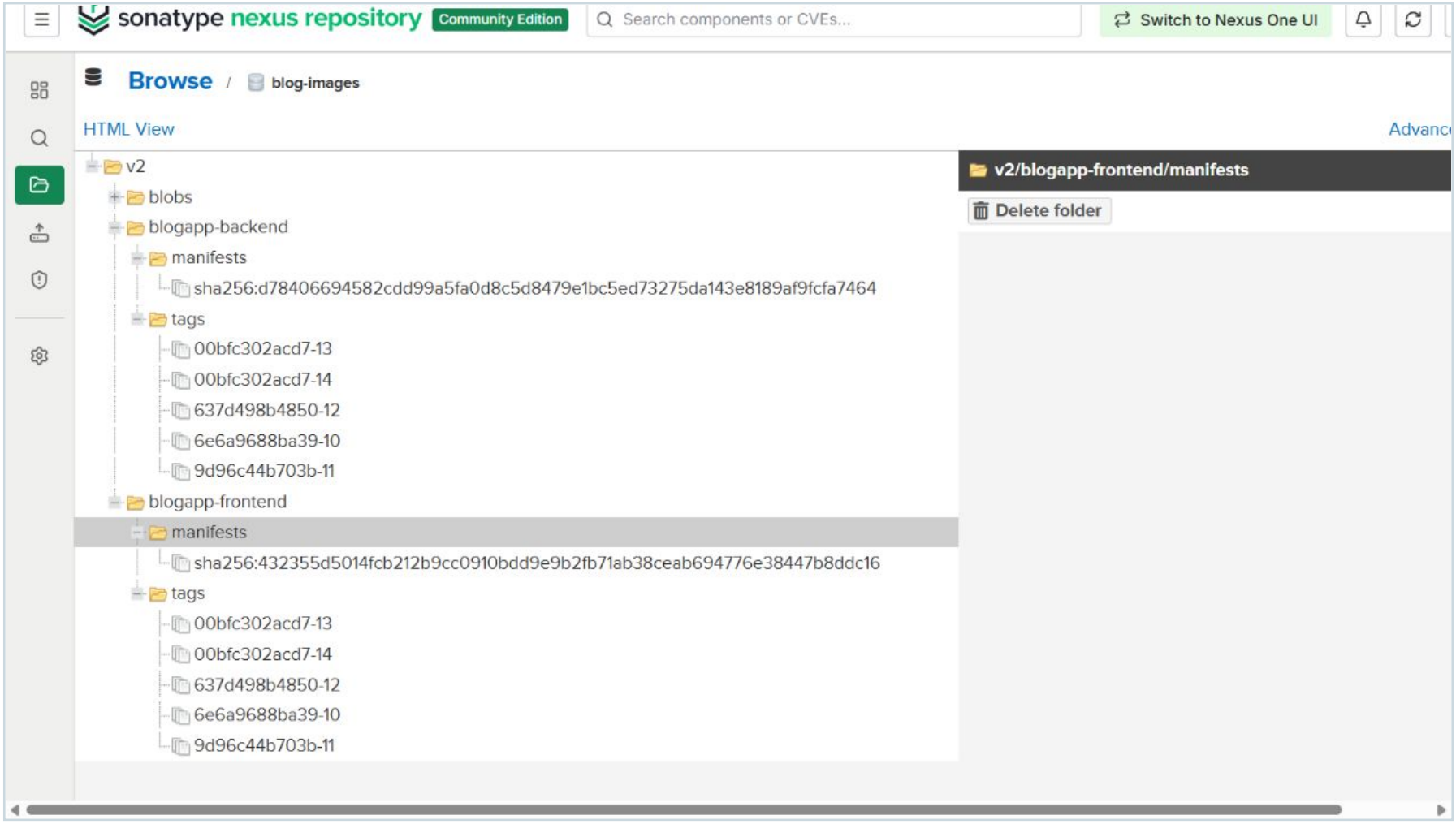
An isolated image containing known vulnerabilities was scanned with the same HIGH/CRITICAL enforcement settings. Its failed scan prevented the deployment stage from running.



Separate deliberately vulnerable Docker image: Trivy-Image-Gate-Negative #1. Image scan failed and deployment was skipped. Full vulnerability JSON is attached.

Versioned images and matching digests in Nexus

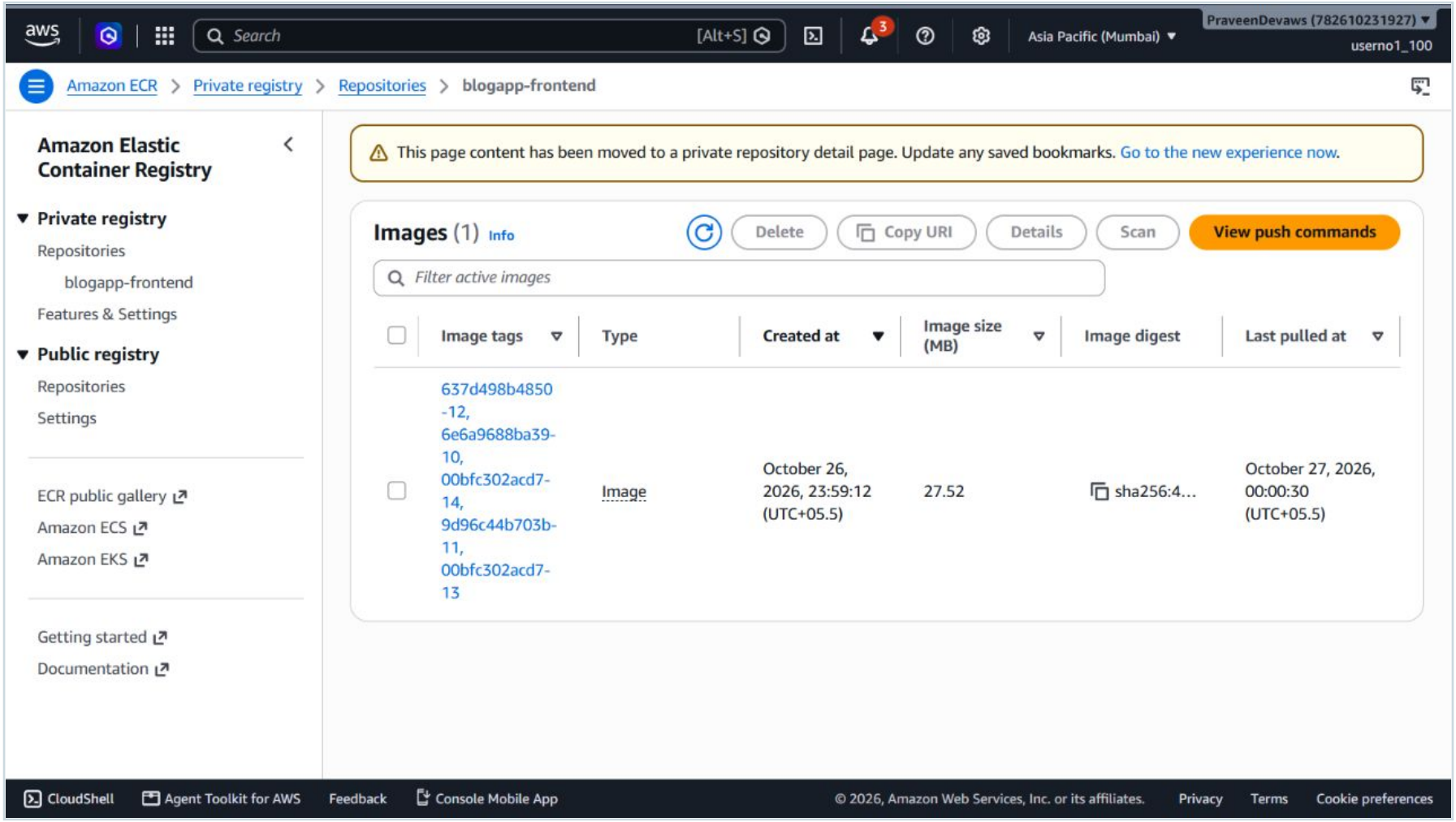
Nexus stores both frontend/backend image versions, with redeployment disabled, and a committed source archive. A repository-scoped deployment account publishes through a private TLS endpoint. Nexus now uses a protected project-specific encryption key; its health checks passed.



Nexus image tree shows 00bfc302acd7-14 for both components and the same content digests used by ECR. Raw Nexus component/archive metadata is attached.

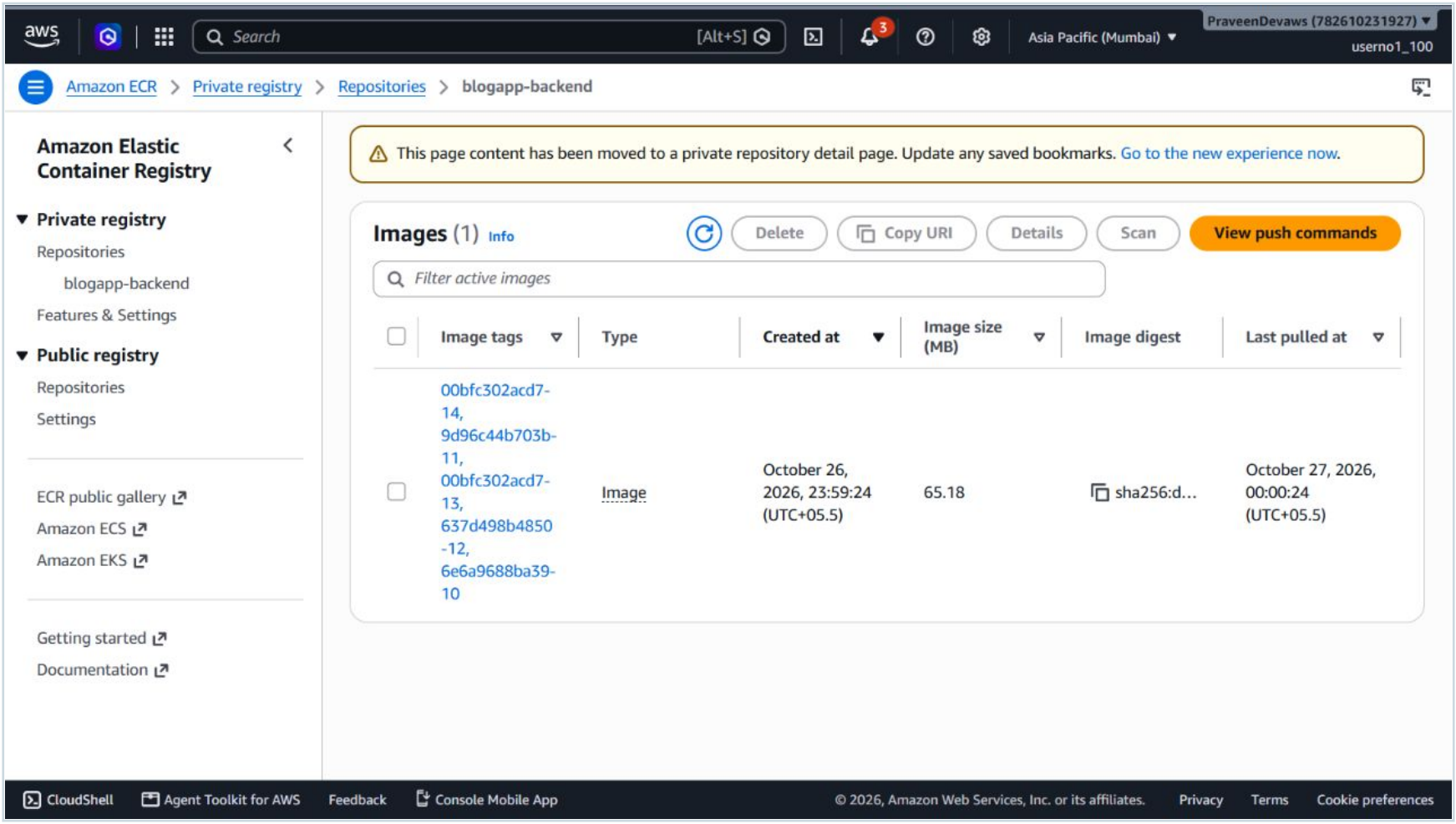
Versioned frontend image in the private ECR registry

ECR repositories use encryption, immutable tags and scan-on-push. EKS pulls from ECR. Different release tags may reference identical cached application content; the tags remain immutable and traceable to their build/source records.



ECR frontend repository includes release tag 00bfc302acd7-14. Exact digest and registry record: raw/E05-ecr-final-images.json.

Versioned backend image in the private ECR registry

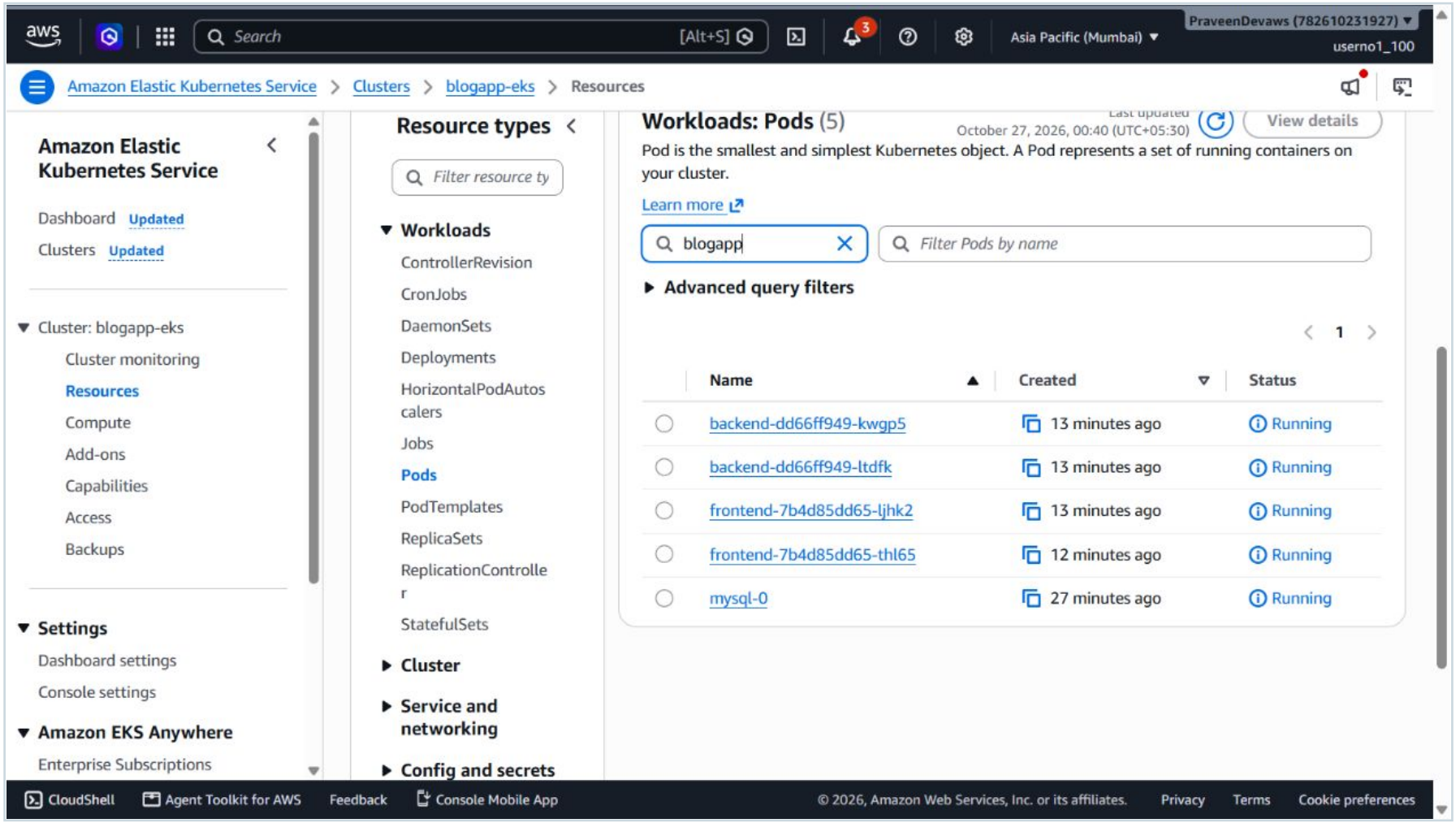


ECR backend repository includes release tag 00bfc302acd7-14. The screenshot size is registry-compressed storage, not the uncompressed Docker runtime size.

Component	Digest shared by Nexus and ECR
frontend	sha256:432355d5014fcb212b9cc0910bdd9e9b2fb71ab38ceab694776e38447b8ddc16
backend	sha256:d78406694582cdd99a5fa0d8c5d8479e1bc5ed73275da143e8189af9fcfa7464

Two frontend pods, two backend pods and MySQL

Five application pods are Running. Frontend/backend Deployments each maintain two replicas. Readiness and liveness probes, requests/limits, rolling update maxUnavailable=0/maxSurge=1, PDBs and HPA definitions are included in k8s/. All three application Services remain internal.



Actual AWS EKS namespace pod view. Kubernetes configuration/runtime snapshots are attached under E07 and E03 in raw/.

Deployed image tags and content digests in EKS

The immutable build #14 images were deployed for release validation. This capture shows two Ready replicas per Deployment with tag 00bfc302acd7-14 and pod image IDs matching the recorded ECR/Nexus content digests.

Live operator command console / blogapp

Real subprocess output streamed from kubectl against Amazon EKS. No saved output is loaded. Secret values are excluded.

Run images check — Completed

```
Executed: 2026-10-26T19:53:35.477284+00:00
Context: personal AWS account / ap-south-1 / blogapp-eks / namespace blogapp

$ kubectl get deploy -n blogapp -o wide
NAME       READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                                                                    SELECTOR
backend    2/2     2            2           83m   backend      782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14    app=backend
frontend   2/2     2            2           83m   frontend     782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-frontend:00bfc302acd7-14   app=frontend
[exit 0]

$ kubectl get pods -n blogapp -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[*].image,IMAGE_ID:.status.containerStatuses[*].imageID
NAME                        IMAGE                                                                    IMAGE_ID
backend-dd66ff949-2sh2t     782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14    782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-
backend@sha256:d78406694582cdd99a5fa0d8c5d8479e1bc5ed73275da143e8189af9fcfa7464
backend-dd66ff949-lqdmx     782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14    782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-
backend@sha256:d78406694582cdd99a5fa0d8c5d8479e1bc5ed73275da143e8189af9fcfa7464
frontend-7b4d85dd65-btn57   782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-frontend:00bfc302acd7-14   782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-
frontend@sha256:432355d5014fcb212b9cc0910bdd9e9b2fb71ab38ceab694776e38447b8ddc16
frontend-7b4d85dd65-wrgq6   782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-frontend:00bfc302acd7-14   782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-
frontend@sha256:432355d5014fcb212b9cc0910bdd9e9b2fb71ab38ceab694776e38447b8ddc16
mysql-0                     mysql:8.4
docker.io/library/mysql@sha256:76f0358884cb92d9a086d24e5671c38048d3fef40f2588703d73eb55392d6b69
[exit 0]


All checks in this live execution completed.

Process exit code: 0
```

Live kubectl get deploy -o wide and pod image IDs. Full output: raw/S-images-live-console.txt.

Replicas, rolling strategy, probes and resources

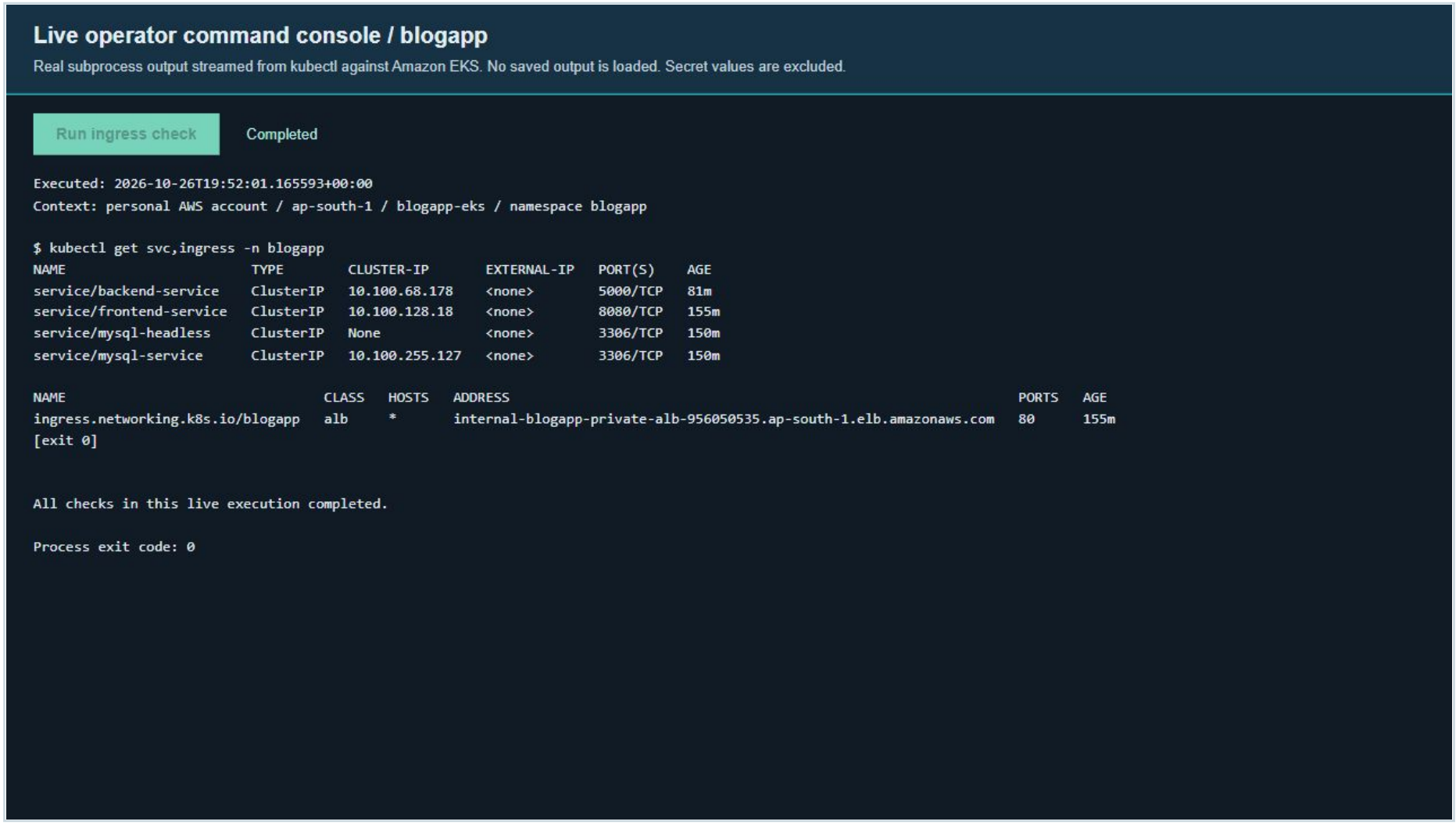
Actual kubectl describe deployment backend output, through its first 42 lines. The selected lines include all requested production fields and Secret key references without secret values.

```
$ kubectl describe deployment backend -n blogapp  [first 42 lines]
Name:                   backend
Namespace:              blogapp
CreationTimestamp:      Tue, 27 Oct 2026 00:00:23 +0530
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 9
Selector:               app=backend
Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  0 max unavailable, 1 max surge
Pod Template:
  Labels:  app=backend
  Containers:
   backend:
    Image:           782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14
    Port:            5000/TCP (http)
    Host Port:       0/TCP (http)
    SeccompProfile:  RuntimeDefault
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:      100m
      memory:   128Mi
    Liveness:   http-get http://:5000/healthz delay=0s timeout=1s period=10s #success=1 #failure=3
    Readiness:  http-get http://:5000/readyz delay=0s timeout=1s period=5s #success=1 #failure=3
    Startup:    http-get http://:5000/healthz delay=0s timeout=1s period=5s #success=1 #failure=30
    Environment Variables from:
      backend-config  ConfigMap  Optional: false
    Environment:
      DB_PASSWORD:              <set to the key 'DB_PASSWORD' in secret 'blog-secrets'>  Optional: false
      JWT_SECRET:               <set to the key 'JWT_SECRET' in secret 'blog-secrets'>   Optional: false
    Mounts:                     <none>
  Volumes:                      <none>
  Topology Spread Constraints:  kubernetes.io/hostname:ScheduleAnyway when max skew 1 is exceeded for selector app=backend
  Node-Selectors:               <none>
```

Two desired/updated/available replicas; maxUnavailable=0, maxSurge=1; liveness/readiness/startup probes; CPU and memory requests/limits. Raw: S-production-live-console.txt.

Internal Services and an ALB-backed Ingress

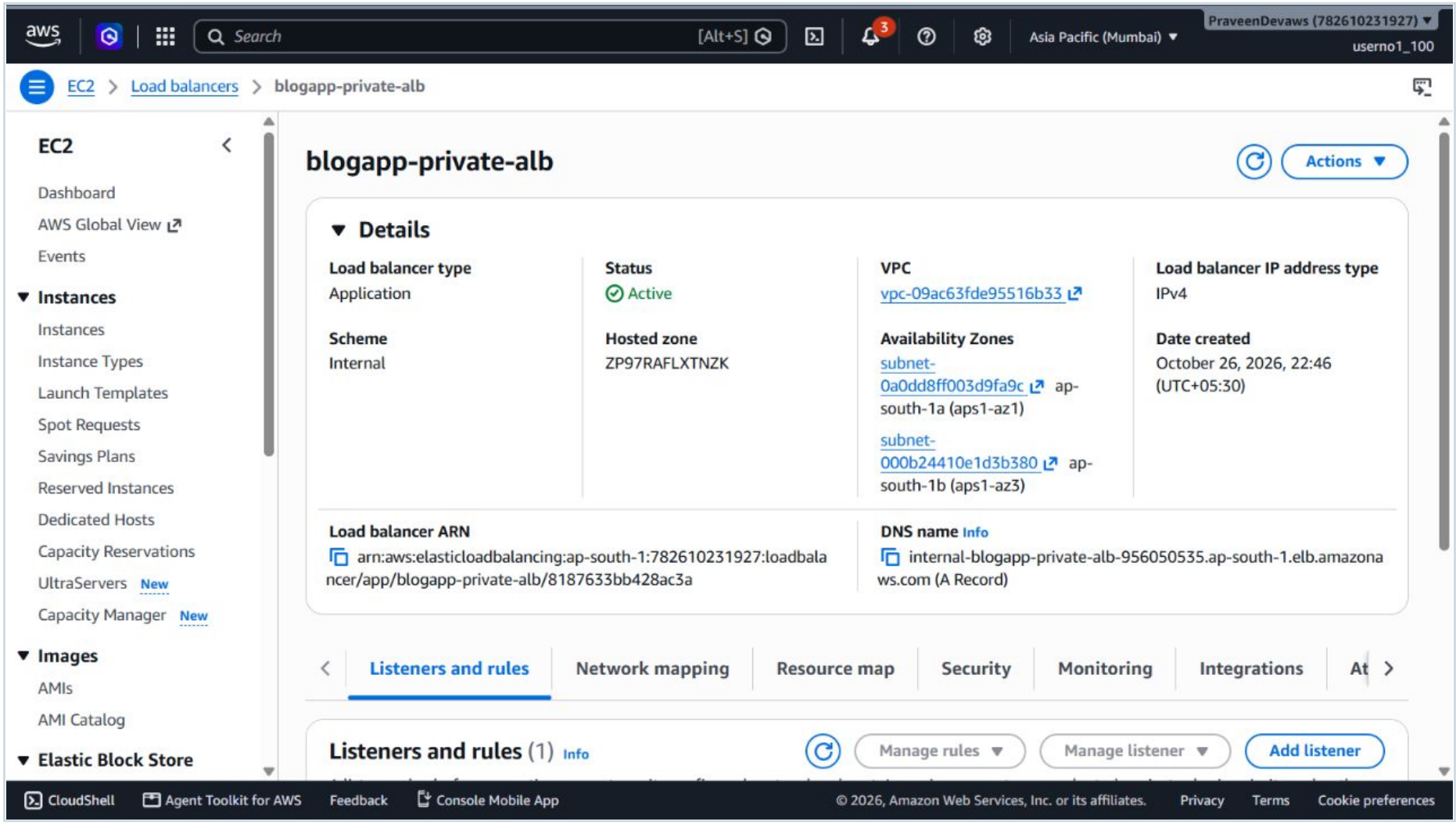
The application uses ClusterIP Services. The alb Ingress publishes an internal ALB address, so its address is not directly internet-accessible. Public access is provided through API Gateway and its VPC Link.



Actual kubectl get svc,ingress -n blogapp. The Ingress address matches the private ALB shown next.

The Ingress ALB is Active and Internal

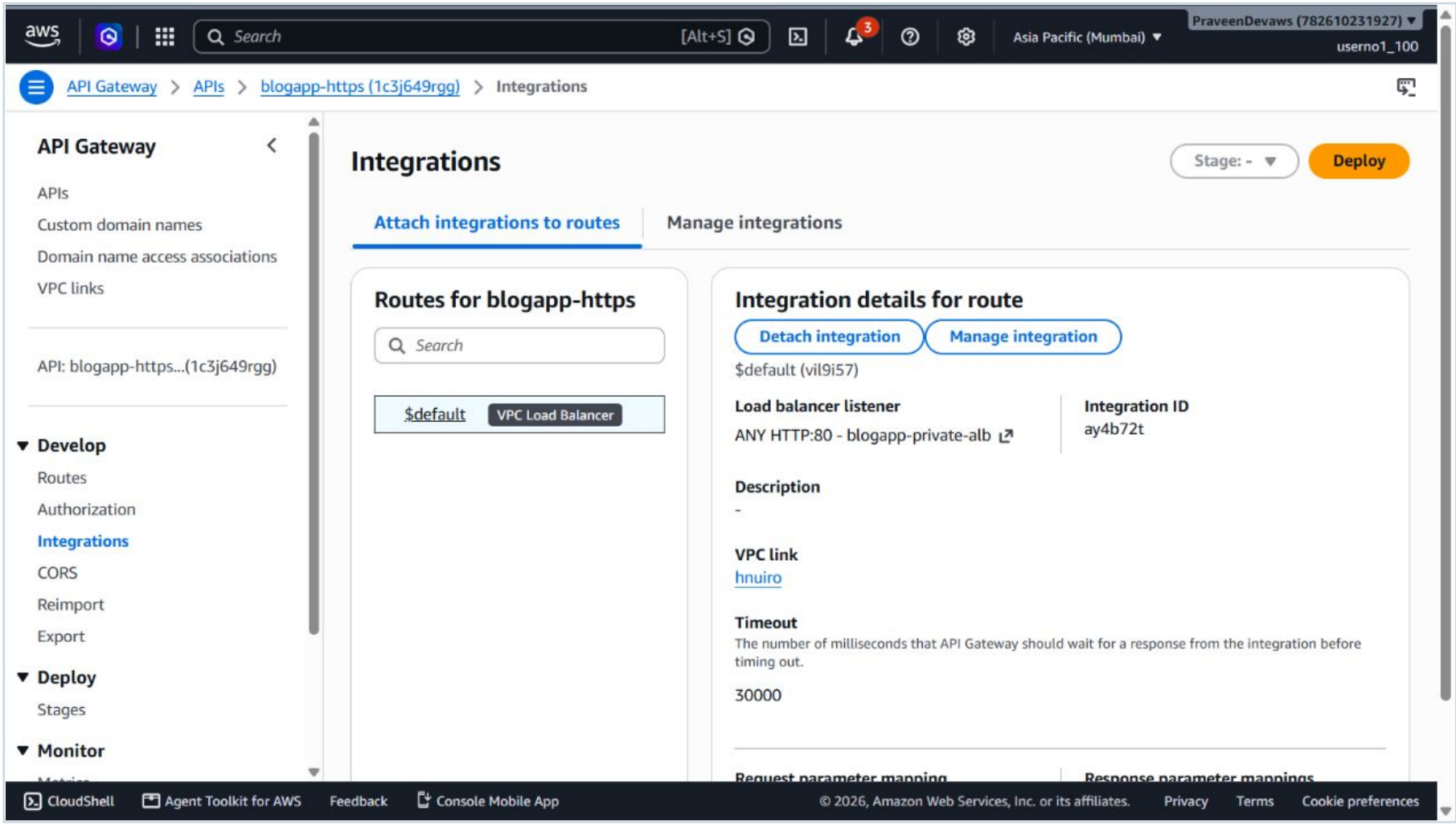
AWS EC2 console details show the application load balancer, its Internal scheme, Active state, private VPC and two Availability Zones. Listener HTTP:80 is the private origin hop.



Actual AWS console: blogapp-private-alb. Public TLS terminates at API Gateway; the VPC Link-to-ALB hop uses HTTP.

API Gateway routes to the private ALB listener

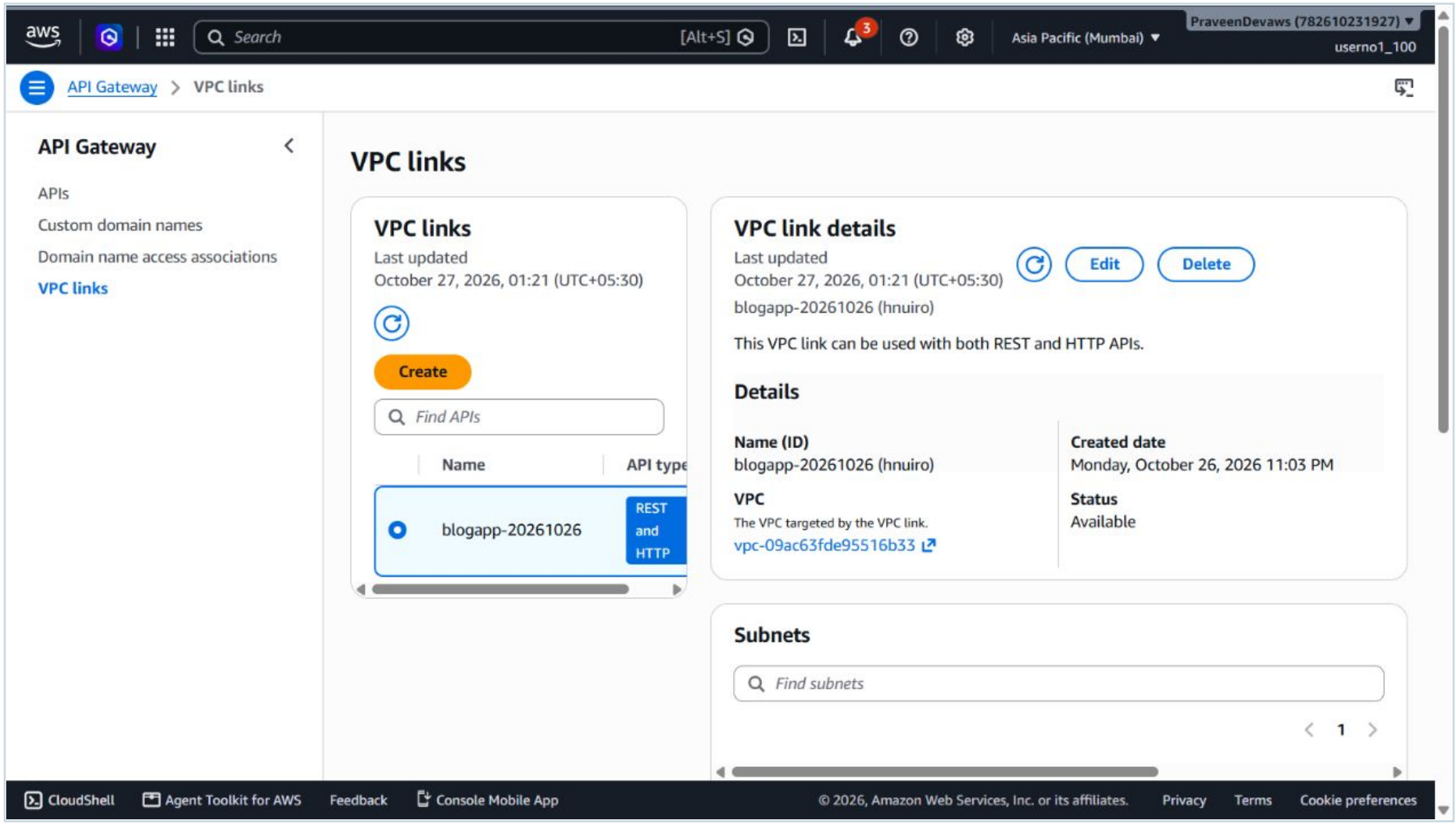
The public HTTPS application URL is https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com. The default route integration identifies the blogapp-private-alb listener and VPC Link hnuiro. The complete executed path is API Gateway -> VPC Link -> internal ALB -> EKS Ingress rules -> frontend/backend Services.



Actual API Gateway console integration details for blogapp-https (1c3j649rgg).

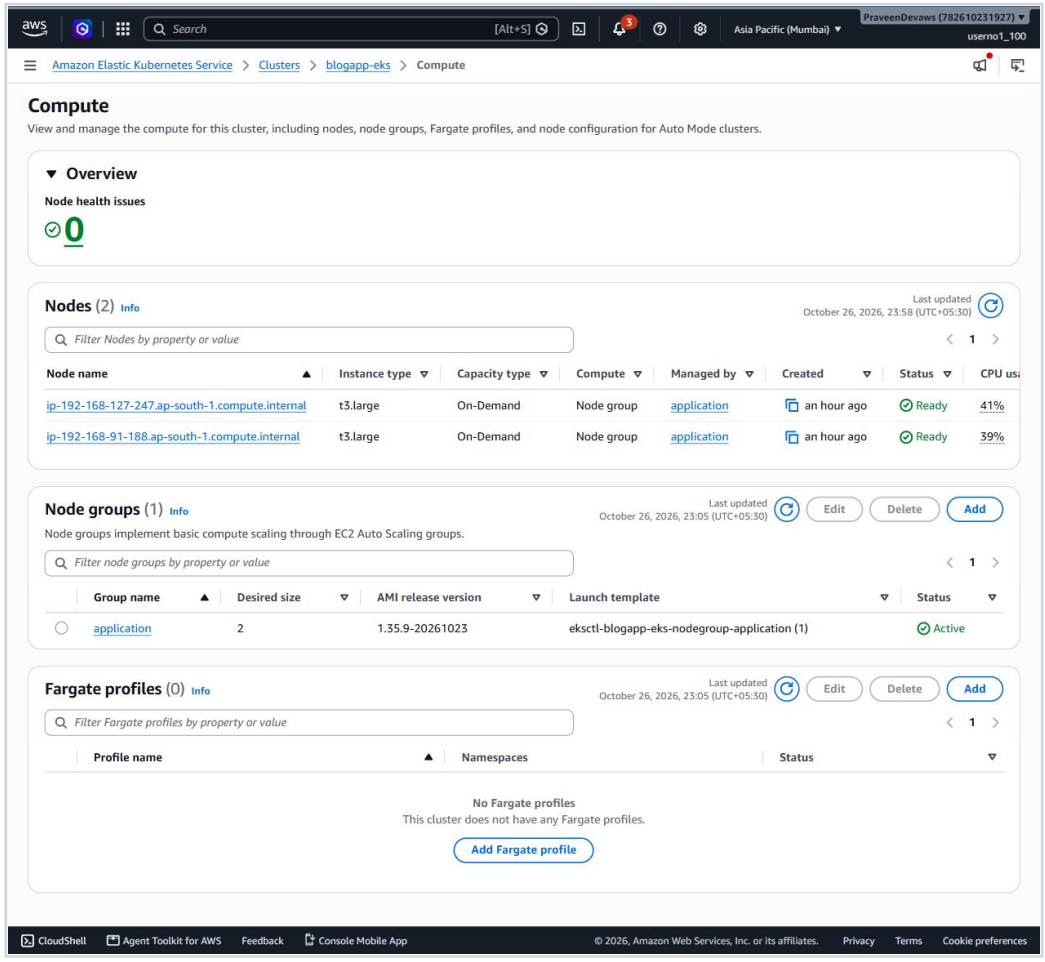
The VPC Link is Available

The VPC Link details identify blogapp-20261026 (hnuiro), its Available state and the same project VPC used by the internal ALB.



Actual AWS API Gateway VPC Link console. The raw Kubernetes Service/Ingress output and AWS console screenshots establish each routing layer.

Healthy worker nodes across two Availability Zones



Two t3.large workers in the managed application node group; zero node health issues. The full-resolution screenshot is included in the evidence folder.

The controller and managed add-ons were installed and verified, including EBS CSI permissions, metrics-server, ALB controller and VPC CNI policy enforcement. The cluster public API is restricted to a single operator CIDR; CI uses private connectivity.

Pod recreation retained data; isolated restore matched

Persistence check	Before	After
Database pod UID	877364e1-3435-4244-a544-2248d9224edf	1fc06f12-c7d4-4693-86e5-1e2b4e647764
Persistent volume	pvc-f58922e6-d455-4c9d-8306-33fa70441279	pvc-f58922e6-d455-4c9d-8306-33fa70441279
Stored post id	3	3
Stored comments	1	1
Recovery time	Replacement pod + public read-back	28.98 seconds

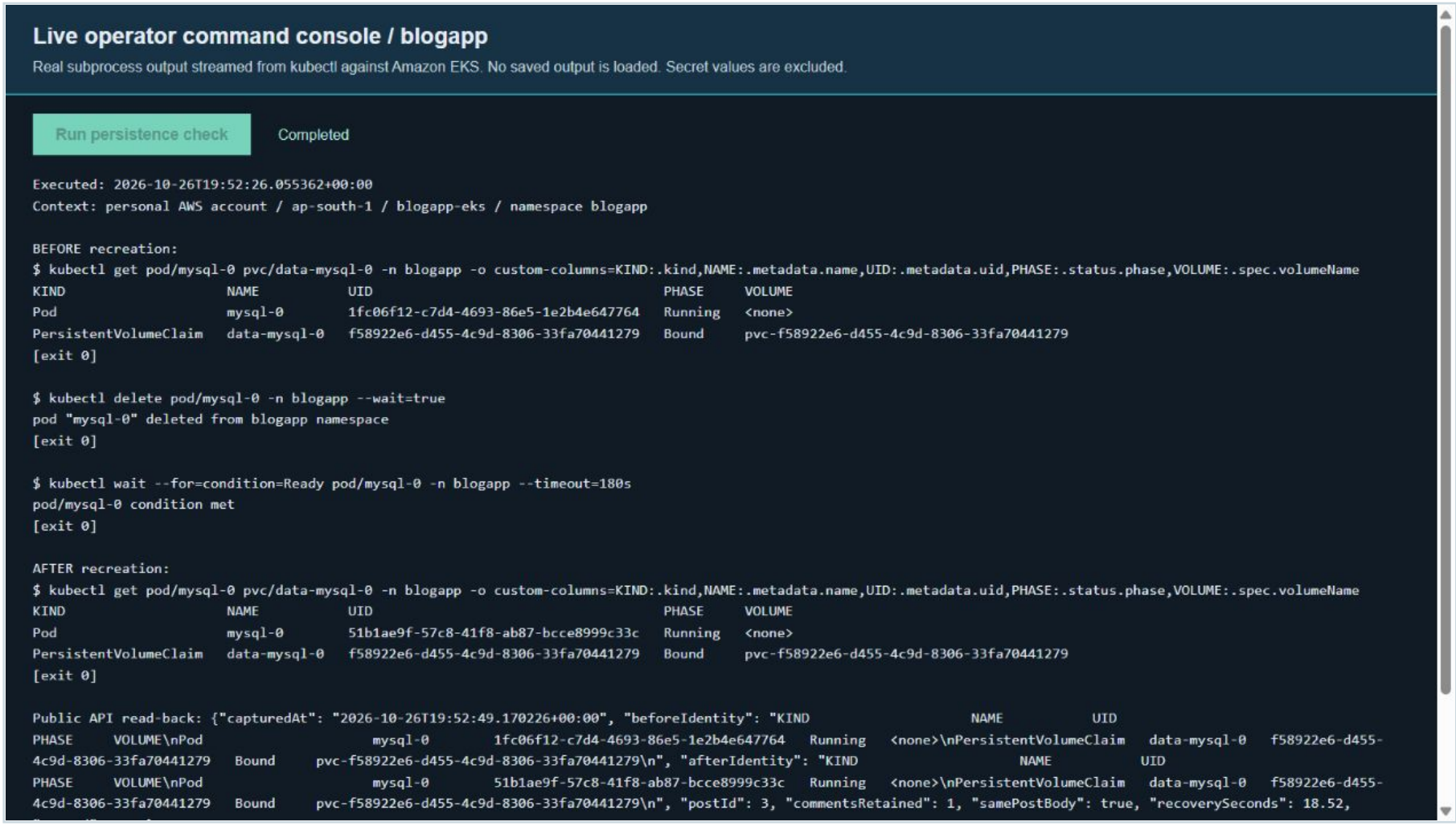
The pod UID changed while the volume and application content stayed the same. This was an intentional single-database maintenance test; the 28.98-second recovery is not a claim of zero-downtime database high availability.

Backup / restore	Source blogdb	Isolated blogdb_restore_proof
Users	3	3
Posts	5	5
Comments	5	5

The 7,849-byte logical backup was restored into a separate database. Its SHA-256 is 50b4581a775b4b6c5db6c0a2a5ed01e87c3ee23793ca33c74e45c2ce5b6ad11a. The SQL backup contains private account data and is retained privately. Later smoke runs add records; the table counts above are the captured restore snapshot.

A new MySQL pod retained the same PVC and content

During the persistence validation run, the pod UID changed from 1fc06f12-c7d4-4693-86e5-1e2b4e647764 to 51b1ae9f-57c8-41f8-ab87-bcce8999c33c. PVC UID and EBS volume stayed unchanged. Public API read-back retained post id 3, its body and one comment; recovery was 18.52 seconds.



Before/delete/wait/after command execution. Run result: raw/S-persistence-repeat.json; full command output: S-persistence-live-console.txt.

196 successful requests during a commit-triggered rollout

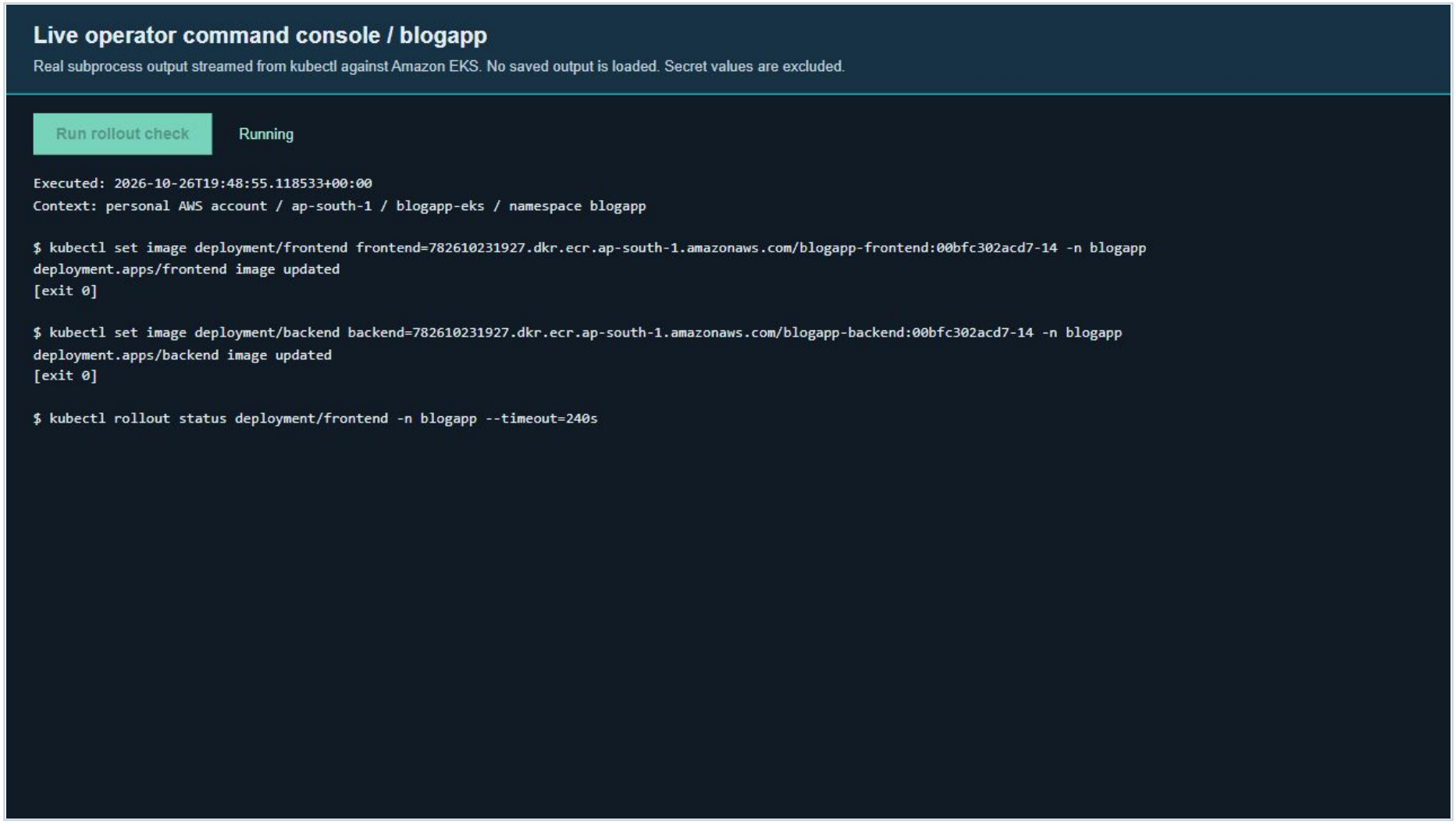
Measured result	Value
Release under continuous sampling	BlogApp/main #12 - SUCCESS; source 637d498b4850cb8baac277fd4286180ca7f14f86
Public requests sampled	196
HTTP 200 responses	196
Failures	0
Maximum observed request latency	1593.0 ms
Validation data	raw/E10-rolling-traffic-test.json and E10-rolling-traffic-samples.csv

The initial rollout had exposed a transient 502. The implementation then added ALB pod readiness gates, 20-second target deregistration, 25-second frontend preStop, 10-second backend preStop and a 60-second termination grace period. The next commit-triggered rollout passed the measured traffic test.

The 196-request sample supports observed availability during that test. It is not a universal downtime or latency guarantee. Release build #14 subsequently used the same configured rollout controls and passed public smoke validation.

Rolling redeployment of the versioned immutable images

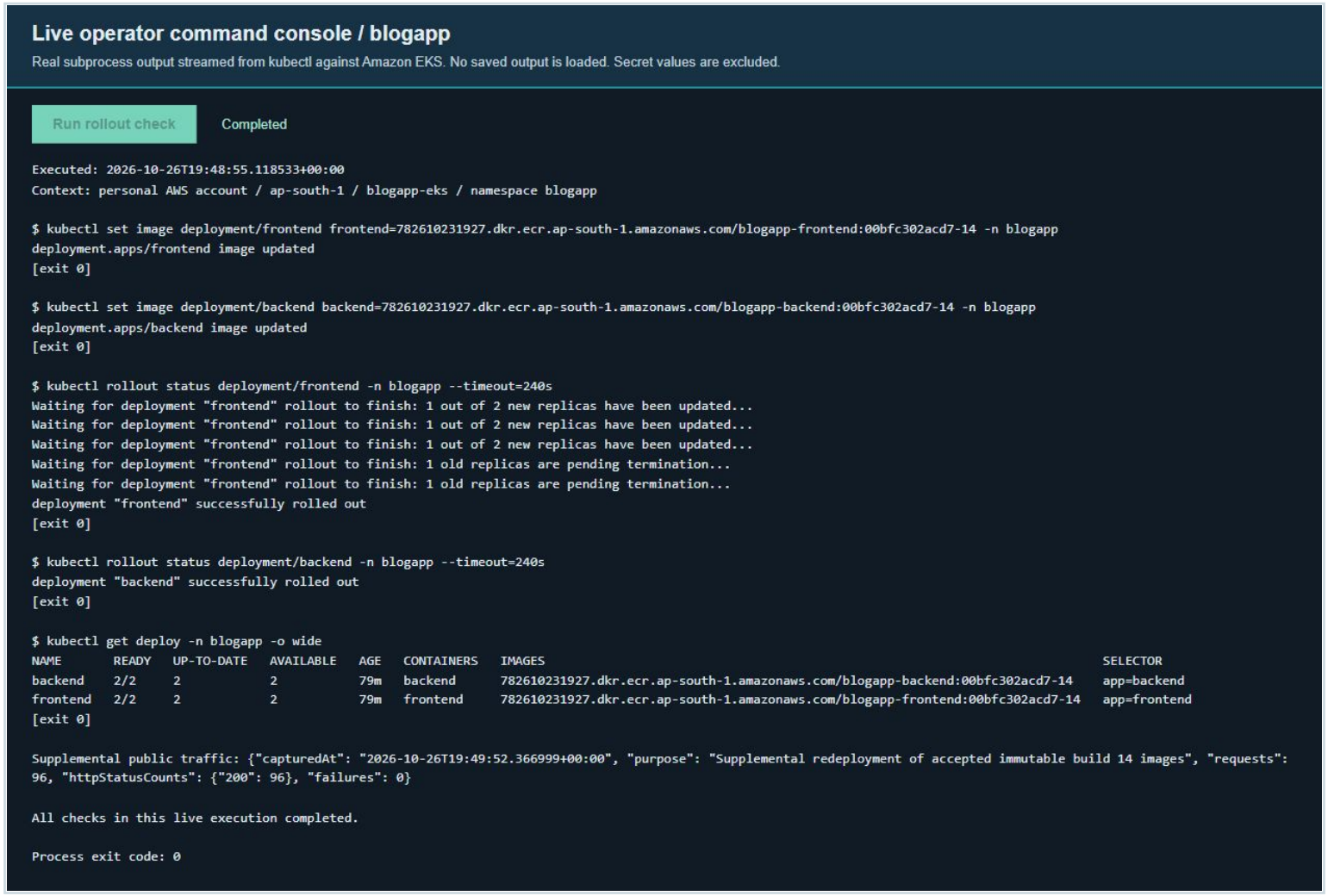
The screenshot captures kubectl rollout status waiting during redeployment of build #14 images. Public API requests were sampled concurrently to measure availability throughout this execution.



Live rollout execution for the immutable frontend and backend image versions.

Both Deployments completed; 96 of 96 requests passed

Frontend and backend completed the rolling deployment successfully. A concurrent client sampled the public /api/posts endpoint throughout the run: 96 HTTP 200 responses and zero failures. Build #12 has a separate 196-request availability measurement.



Live operator command console / blogapp

Real subprocess output streamed from kubectl against Amazon EKS. No saved output is loaded. Secret values are excluded.

Run rollout check — Completed

```
Executed: 2026-10-26T19:48:55.118533+00:00
Context: personal AWS account / ap-south-1 / blogapp-eks / namespace blogapp

$ kubectl set image deployment/frontend frontend=782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-frontend:00bfc302acd7-14 -n blogapp
deployment.apps/frontend image updated
[exit 0]

$ kubectl set image deployment/backend backend=782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14 -n blogapp
deployment.apps/backend image updated
[exit 0]

$ kubectl rollout status deployment/frontend -n blogapp --timeout=240s
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
deployment "frontend" successfully rolled out
[exit 0]

$ kubectl rollout status deployment/backend -n blogapp --timeout=240s
deployment "backend" successfully rolled out
[exit 0]

$ kubectl get deploy -n blogapp -o wide
NAME       READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                                                                    SELECTOR
backend    2/2     2            2           79m   backend      782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-backend:00bfc302acd7-14    app=backend
frontend   2/2     2            2           79m   frontend     782610231927.dkr.ecr.ap-south-1.amazonaws.com/blogapp-frontend:00bfc302acd7-14   app=frontend
[exit 0]

Supplemental public traffic: {"capturedAt": "2026-10-26T19:49:52.366999+00:00", "purpose": "Supplemental redeployment of accepted immutable build 14 images", "requests":
96, "httpStatusCounts": {"200": 96}, "failures": 0}

All checks in this live execution completed.

Process exit code: 0
```

Rollout completion output with versioned image tags. Traffic results: raw/S-rollout-traffic.json; command record: S-rollout-live-console.txt.

Configuration values are separated from images

Input	Production mapping	Evidence
frontend/.env.example: VITE_API_URL=/api	Relative browser API path; Nginx routes to backend-service:5000	Frontend bundle + Nginx config
PORT / DB_HOST / DB_PORT / DB_USER / DB_NAME	backend-config ConfigMap	raw/E11-config-map.json
CORS_ORIGIN	Actual public HTTPS application origin	raw/E11-config-map.json
DB_PASSWORD / JWT_SECRET	blog-secrets Secret; generated once; Secrets Manager copy	Only metadata and references exported
MYSQL_ROOT_PASSWORD	Database Secret reference; not in schema or source	k8s/mysql.json
Pipeline credentials and Nexus encryption key	Protected CI host/Jenkins credential storage; TLS private CA trust	No key/password material in the public package

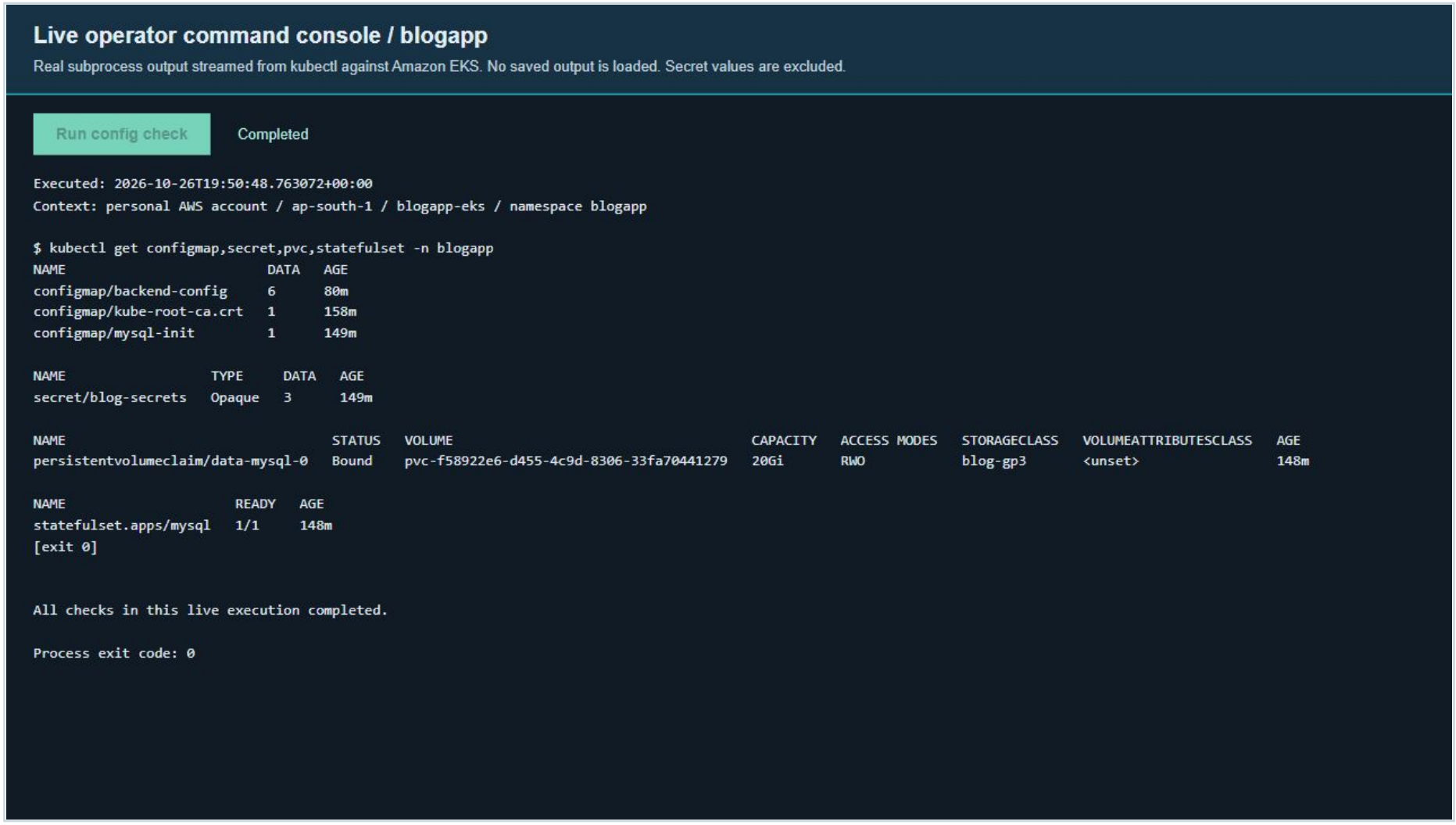
The .env.example secret fields are empty. The real local environment, AWS session files, CI passwords, TLS private keys and SQL backup are excluded from delivery. Secret values were injected through protected API/stdin operations and were not embedded in Dockerfiles or Kubernetes source manifests.

```
NAME           TYPE     CREATED
blog-secrets   Opaque   2026-10-26T17:21:37Z
```

Source and exported evidence were reviewed for credential patterns before publication. This is a scoped credential review, not a claim that automated pattern matching alone proves the absence of every possible secret.

ConfigMaps, Secret metadata, PVC and StatefulSet

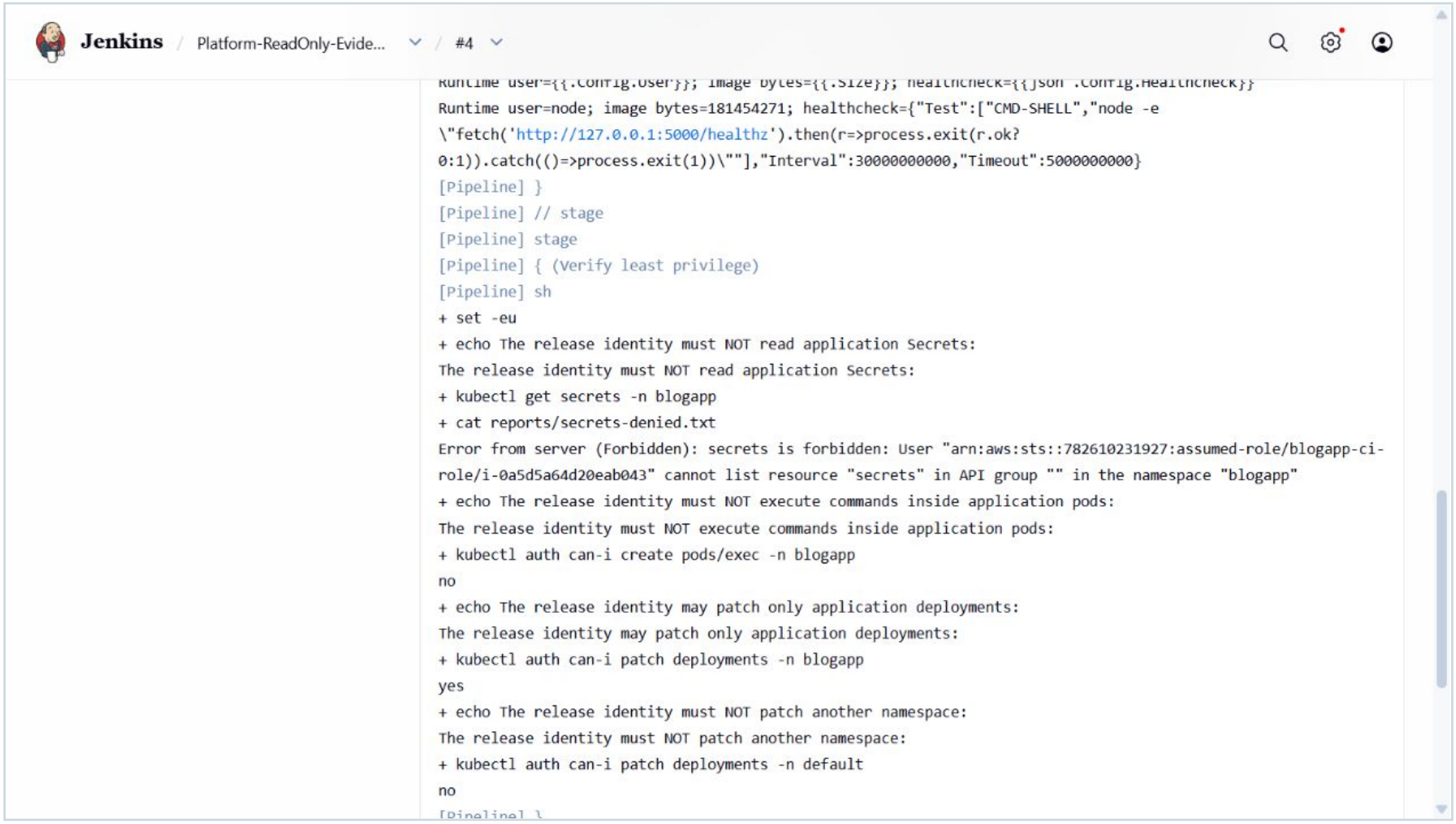
Actual kubectl get configmap,secret,pvc,statefulset -n blogapp. The output displays names, types, data counts, PVC binding/capacity and StatefulSet readiness. It contains no Secret values.



The Secret has three keys; data-mysql-0 is Bound on blog-gp3 with 20 GiB; mysql is 1/1 Ready. Raw: S-config-live-console.txt.

The release role cannot read Secrets or execute in pods

The CI IAM role has ECR authorization plus publishing permissions only for the two application repositories, EKS DescribeCluster for blogapp-eks, and the SSM managed-instance role. Its Kubernetes group is bound to the namespace-scoped blog-deployer Role.



Actual Forbidden response for Secret listing, pod execution denied, application Deployment patch allowed, default-namespace patch denied. The restricted release role was retained.

Intended traffic allowed; unrelated client rejected

Actual test	Expected	Observed
Unrelated pod cannot reach MySQL	Denied	PASS; exit 1; 9.02 s
Unrelated pod cannot reach backend	Denied	PASS; exit 1; 9.84 s
Frontend can reach backend	Allowed	PASS; exit 0; 5.91 s
Backend can reach MySQL	Allowed	PASS; exit 0; 7.5 s

A temporary unrelated busybox client was created in blogapp. TCP access to mysql-service:3306 timed out and backend readiness access was blocked. Existing frontend and backend containers then verified their intended connections. The test client was removed.

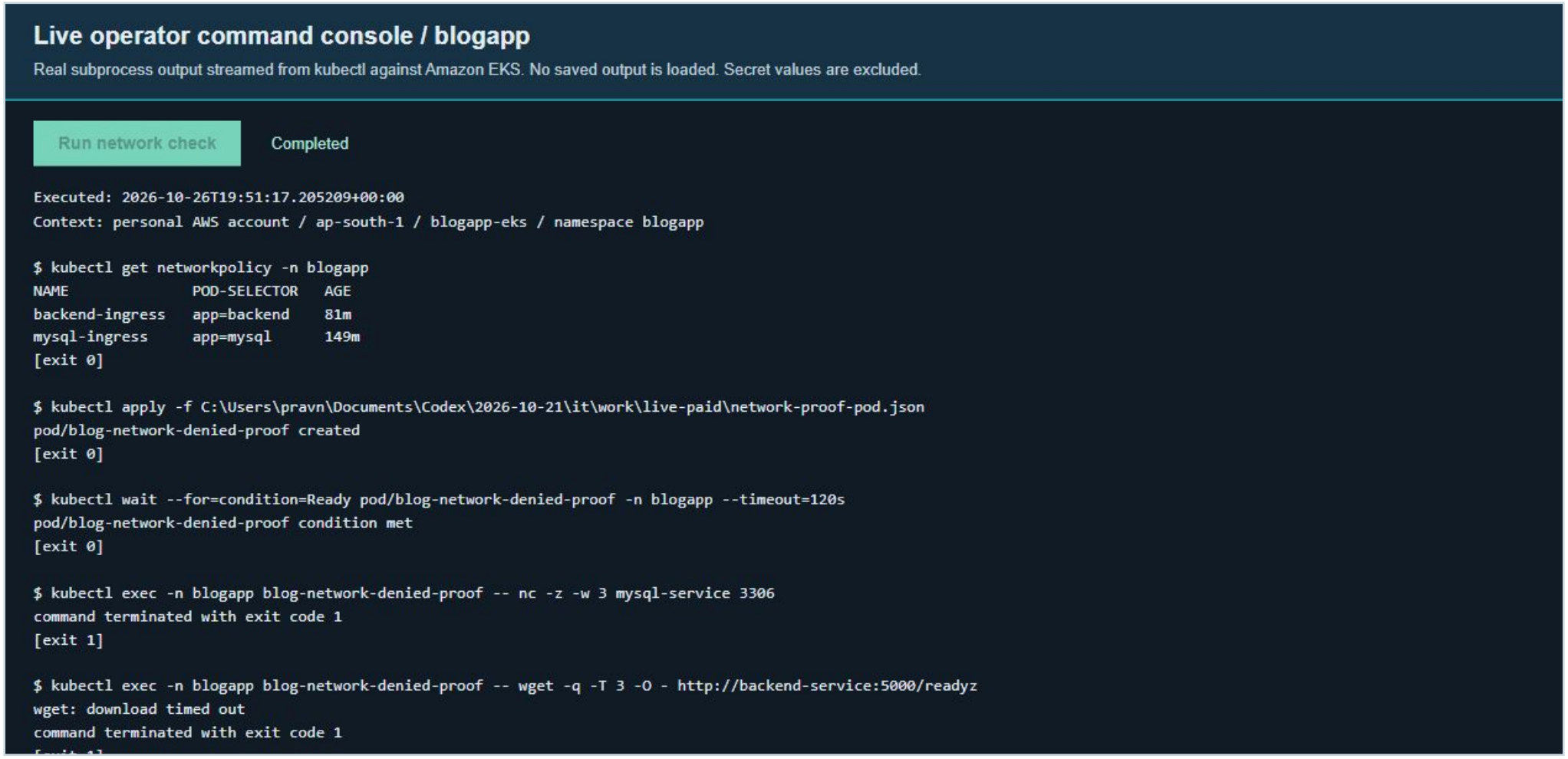
mysql-ingress selects MySQL and allows TCP 3306 only from app=backend pods in the application namespace. backend-ingress allows application traffic from frontend pods and metrics traffic from the monitoring namespace. VPC CNI policy enforcement was enabled and the negative tests confirmed it was effective.

```
Evidence: raw/E12-network-policy-tests.json
Configuration: raw/E12-network-policies.json
CNI and security-group metadata: raw/E11-security-platform.json
```

Private CI has no inbound security-group rules. The internal ALB accepts port 80 from the API Gateway VPC Link security group, while the application database is not externally exposed.

NetworkPolicy inventory and real connection tests

The policies were listed and four connection checks were executed. The unrelated client received exit 1 for both forbidden destinations; the existing frontend reached backend readiness, and backend opened MySQL TCP successfully. The temporary test pod was removed.



Top detail from the genuine full-page live execution capture: policy inventory, client creation and both denied destinations. Uncropped original and full command record are attached.

The existing frontend and backend can communicate

The lower detail of the same execution shows frontend readiness returning ready and backend-to-MySQL TCP succeeding, followed by temporary-client cleanup. Cropping enlarges the original screenshot for readability; no output has been changed.

```
[exit 0]

$ kubectl exec -n blogapp blog-network-denied-proof -- nc -z -w 3 mysql-service 3306
command terminated with exit code 1
[exit 1]

$ kubectl exec -n blogapp blog-network-denied-proof -- wget -q -T 3 -O - http://backend-service:5000/readyz
wget: download timed out
command terminated with exit code 1
[exit 1]

$ kubectl exec -n blogapp deployment/frontend -- wget -q -T 5 -O - http://backend-service:5000/readyz
{"status":"ready"}
[exit 0]

$ kubectl exec -n blogapp deployment/backend -- node -e const net=require('net');const s=net.createConnection({host:'mysql-
service',port:3306});s.setTimeout(3000);s.on('connect',()=>{console.log('MySQL TCP connection allowed from backend');s.destroy();});s.on('error',
()=>process.exit(1));s.on('timeout',()=>process.exit(1));
MySQL TCP connection allowed from backend
[exit 0]

$ kubectl delete pod/blog-network-denied-proof -n blogapp --wait=false
pod "blog-network-denied-proof" deleted from blogapp namespace
[exit 0]

PASS: unrelated client denied; intended frontend/backend traffic allowed.

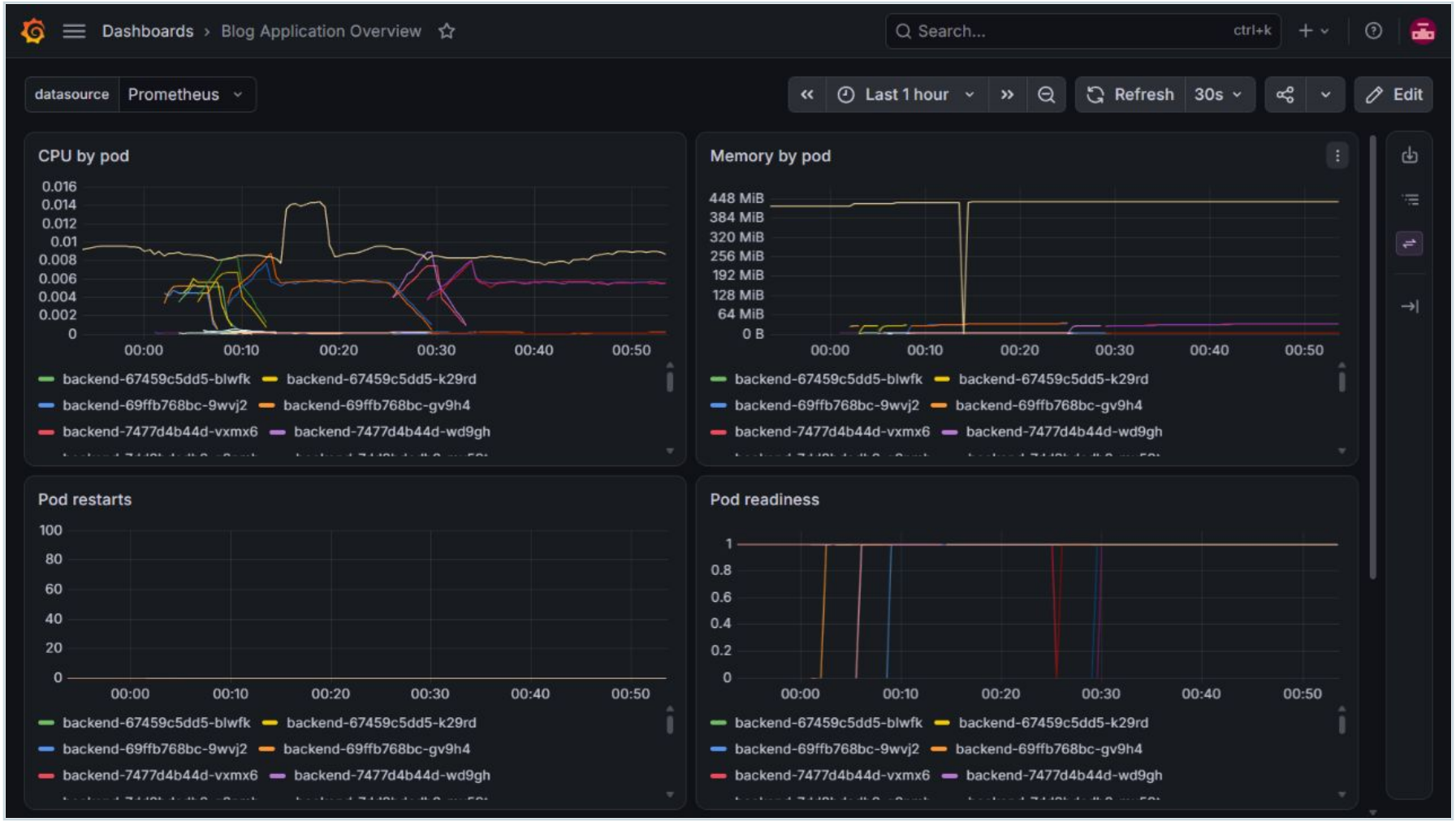
All checks in this live execution completed.

Process exit code: 0
```

Lower detail from S12-live-network-policy-tests.png. Raw: S-network-live-console.txt. The exit 1 results are expected denials; both allowed connection checks exit 0.

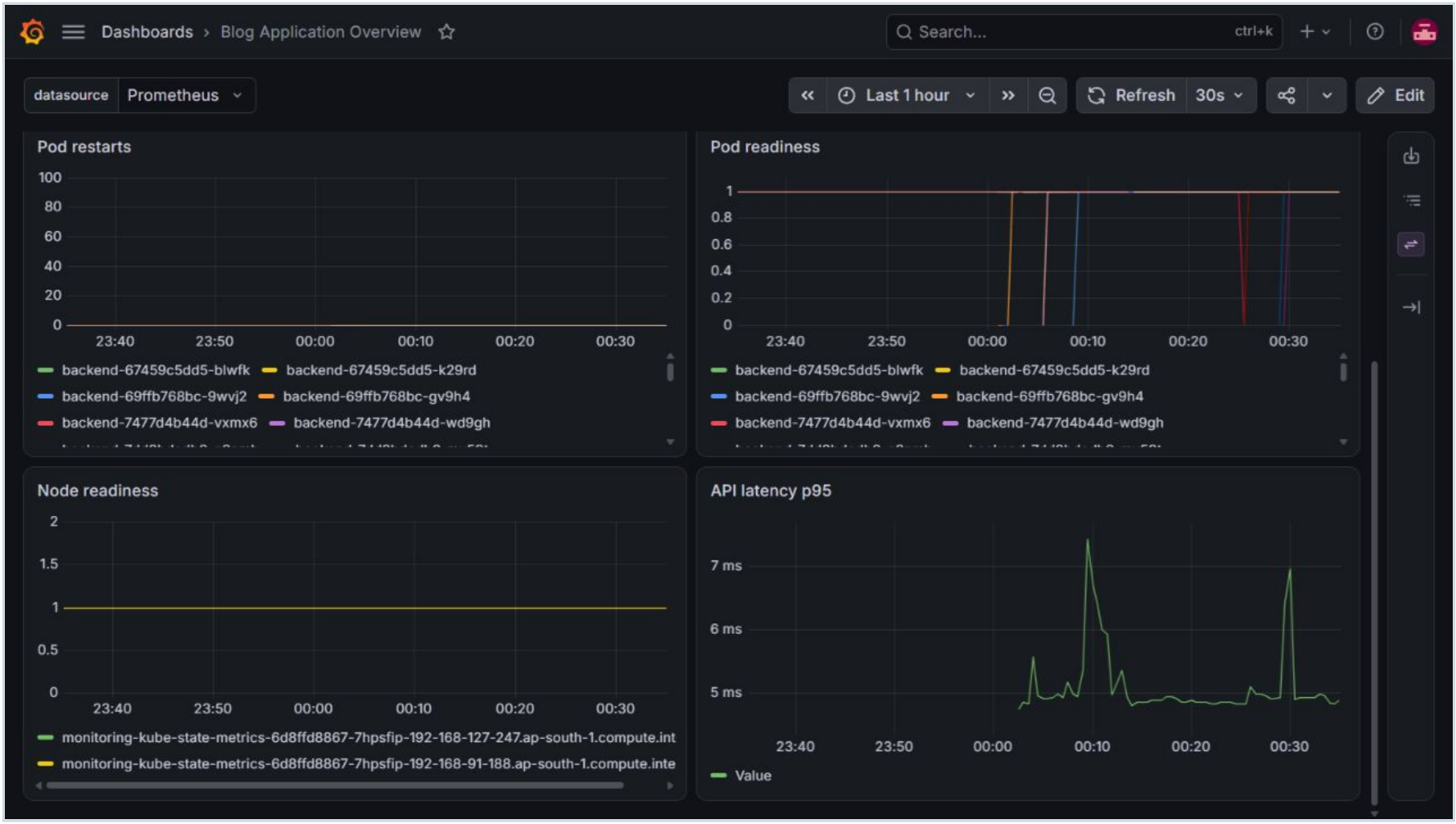
CPU, memory, restarts and readiness graphs

Prometheus, Grafana and Alertmanager run with persistent volumes. The application ServiceMonitor scrapes backend metrics. All six dashboard expressions were queried successfully and returned data. The dashboard includes pod series recorded during rolling releases.



Actual Grafana application dashboard. Query results: raw/E13-monitoring-verification.json.

Node readiness and API latency are populated

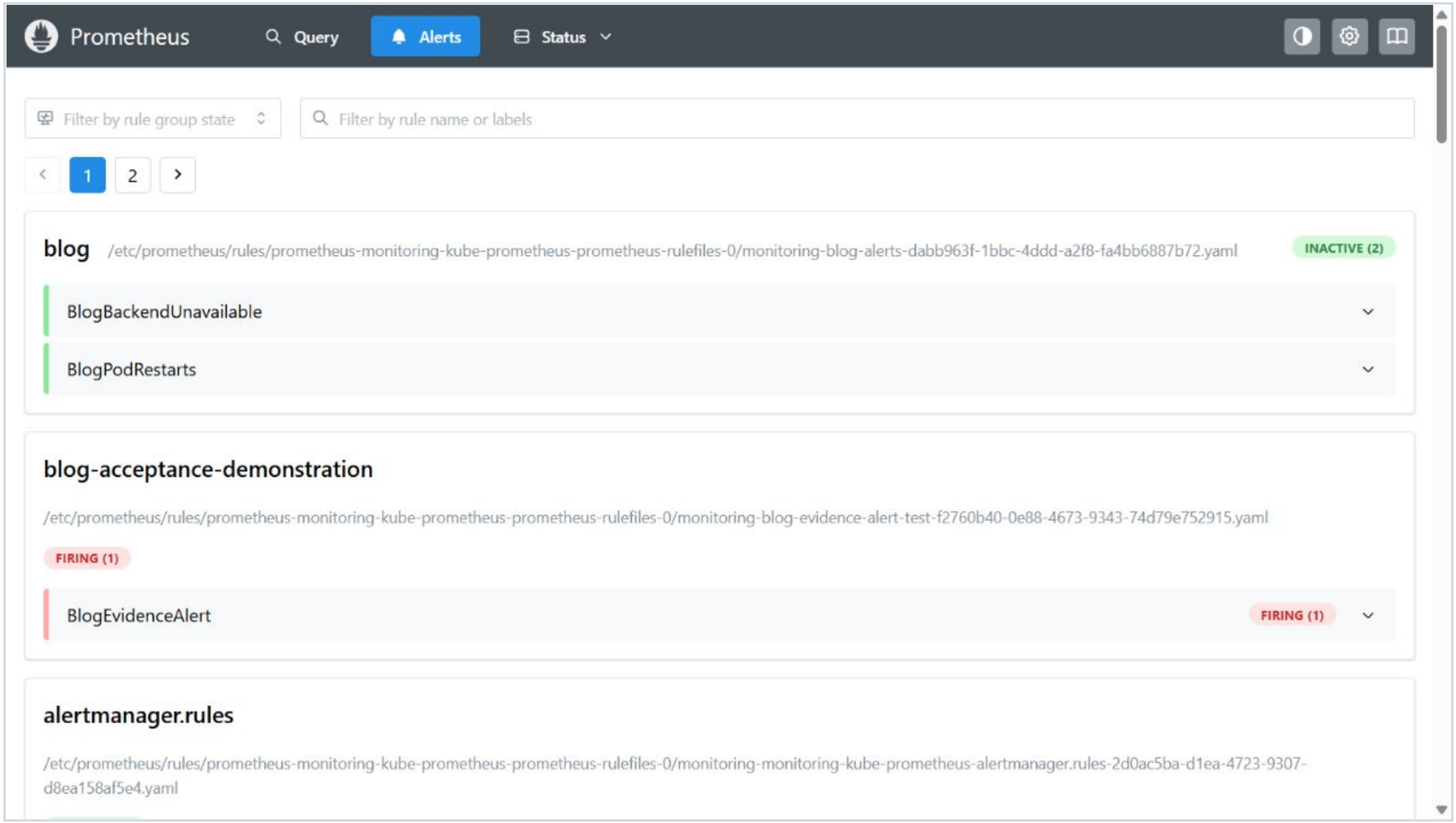


The two Ready-node series overlap at 1. API latency p95 is based on the backend request histogram. These metrics do not represent internet round-trip latency.

Dashboard definitions are in monitoring/dashboard.json. Infrastructure/cluster metrics come from node exporter, kube-state-metrics and kubelet scraping; backend application metrics come from its /metrics endpoint.

A synthetic test alert fired, then was removed

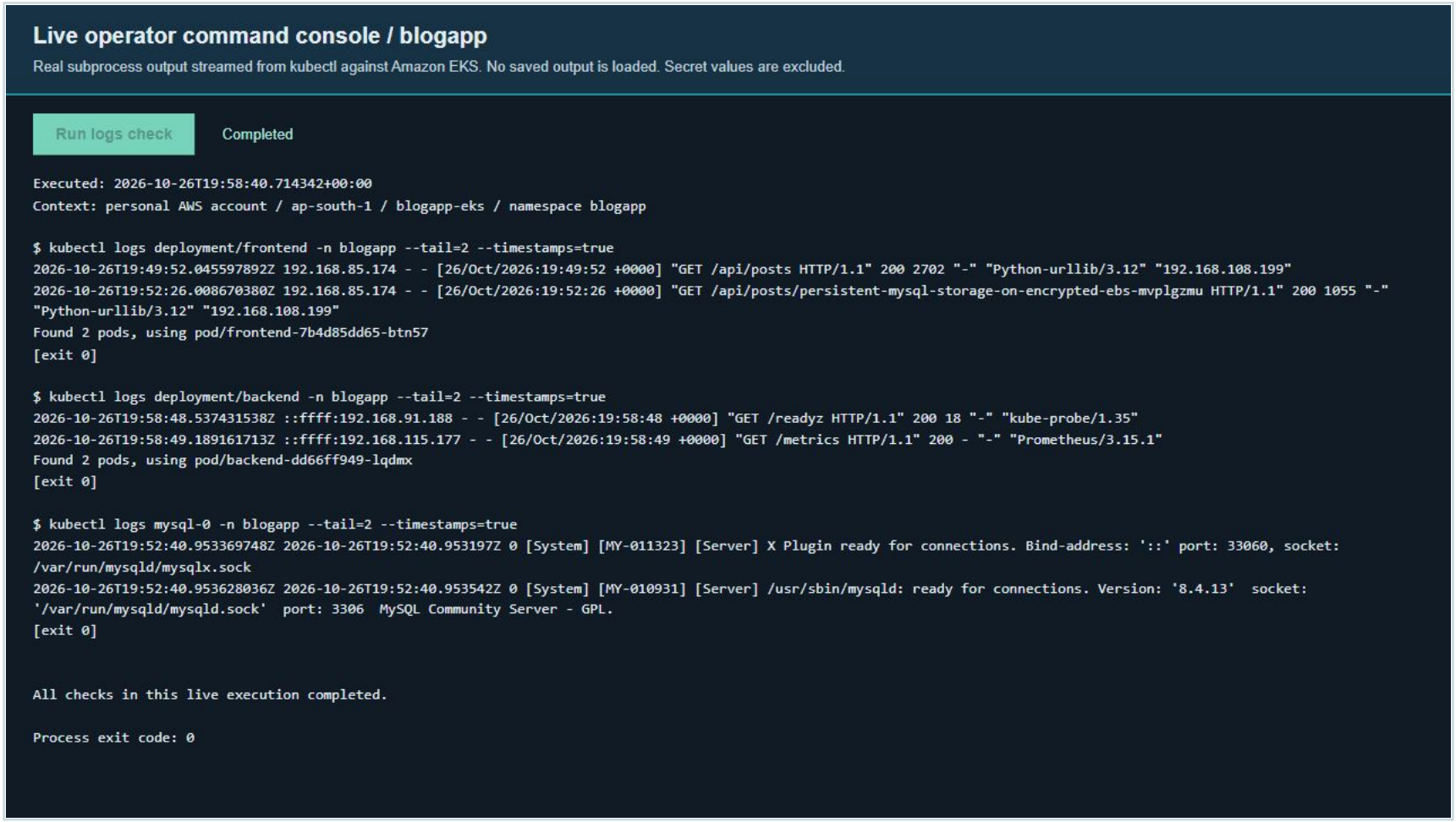
BlogEvidenceAlert deliberately evaluated vector(1) for 15 seconds, with purpose=acceptance-test. The screenshot shows the controlled alert firing while the real blog availability/restart rules are inactive. The test rule was then deleted and zero active synthetic alerts were verified.



Actual Prometheus alert page. Firing and cleanup records: raw/E13-alert-firing-test.json and E13-alert-resolved-test.json. No external email notification is claimed.

Frontend, backend and database log records

Fresh kubectl logs commands were executed for frontend, backend and MySQL with the existing operator identity. This screenshot shows actual subprocess output streamed to the live browser console, without loading a saved transcript. EKS API, audit and authenticator logs are additionally enabled in CloudWatch with seven-day retention.



Live operator browser console, not a native terminal or saved-text viewer. Timestamped frontend, backend and database output: raw/S-logs-live-console.txt. Full command output and timestamped log exports are included in the evidence folder.

An actual CreateContainerConfigError identified a symbolic non-root user; the backend was fixed with numeric UID 1000. Rollout validation exposed a transient 502; readiness/draining was corrected and the measured rerun had zero errors. Both are examples of diagnosing and correcting deployment issues from execution evidence.

Execution evidence index

ID	Proof	Files in cloud-execution-evidence/
E01	Source revision	E01-github-release-source.png; pinned GitHub source commit
E02	Container builds / hardening	E02-deployed-image-hardening.png; E02-live-container-uids.json
E03	Complete pipeline	E03-jenkins-successful-stage-view.png; build-14 console/stages JSON
E04	SonarQube + rejected gate	Sonar passed/rejected screenshots; final and negative gate JSON
E05	Nexus + ECR versions	Nexus and both ECR screenshots; final image, digest and archive metadata
E06	Trivy image gates	Build #14 passing scans + rejected fixture screenshots; exact final image JSON reports
E07	Live EKS workloads	Live tags, describe, Services/Ingress; internal ALB/API Gateway/VPC Link; EKS nodes
E08	Three-tier application	Home + post/comment screenshots; application UAT JSON
E09	Database persistence	Live before/after recreation screenshot; retained post; timestamped persistence records
E10	Rolling traffic	Build #12: 196 requests; build #14 image redeployment: 96 requests and live screenshots
E11	IAM / RBAC / config	Live ConfigMap/Secret/PVC/StatefulSet inventory; denied access; Role/IAM records
E12	Network restrictions	Live policy/test screenshot; positive/negative results; effective policy JSON
E13	Monitoring / alert	Two Grafana screenshots; Prometheus firing screenshot; query/cleanup JSON
E14	Logging	Fresh live kubectl logs screenshot; raw logs; initialization; CloudWatch settings
E15	Backup and restore	Isolated restore result with matching table counts and backup hash

Project links and operational considerations

Application: https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com
Repository: https://github.com/praveenpreni/BlogReact
Document: Cloud_Execution_Report.pdf on GitHub
Release source: 00bfc302acd73933912fd12579b72885081d22cb

Scope / limit	Implementation scope
MySQL and CI availability	Single database instance and single CI host. Persistence/restore is proven; database or CI high availability is not.
Quality coverage	New-code gate passed. Overall coverage remains 40.6%, with 21 existing reliability findings; Community Build has limited injection analysis.
Private tool links	Jenkins/Sonar/Nexus/Grafana localhost views need authenticated operator tunnels. Public access uses the application HTTPS URL.
Release snapshot	Release #14 and immutable tag 00bfc302acd7-14 were recorded. Documentation publication can advance main; tested execution source and image records remain pinned.
Backups and credentials	The SQL backup and all actual passwords, tokens, session files and private keys remain private. Only safe result metadata is attached.
Operating costs and cleanup	AWS charges accrue while resources are running. Decommission the environment at the end of its required operating period, including retained EBS volumes, NAT, CI, EKS, API Gateway, registries, IAM and logs.

The evidence package contains execution screenshots, pipeline output, Kubernetes resource records, scan reports and validation results for the deployed environment.